Grounding Graph Inference with Data Constraints

ABSTRACT

Machine learning models have been routinely queried to retrieve knowledge from graphs via an inference process. Responsible data systems advocate the generation of high-quality interpretations to allow data scientists to track and understand the model output, typically by generating critical subgraphs that are responsible for them. However, such subgraphs may still be incomplete or misleading when they are misaligned with real-world evidence or derived from incomplete graphs. This paper introduces a novel approach that can generate graph interpretations that are simultaneously grounded by evidence suggested by graph data constraints. (1) We formulate *grounded witnesses* as a bi-level interpretation object, consisting of a witness subgraph that explains a graph inference result and a corresponding grounded graph that aligns the witness with user-specified data constraints. (2) We propose quality measures for grounded witnesses, and formulate the problem of witness generation under constraints. We establish the hardness results for the problem. (3) We develop two practical algorithms for grounded witness generation. The first selectively "reverse" and enforce data constraints to make witness graphs grounded by ensuring the co-occurrence of antecedents and consequents of the constraints. The second efficiently extracts arborescences that satisfy the data constraints. Using real-world and synthetic benchmarks, we verify that our algorithms efficiently extract well-grounded interpretations and scale well with large graphs. We also demonstrate their applications in molecular property analysis and cyber event detection.

1 INTRODUCTION

Machine learning models such as graph neural networks (GNNs) [24] have shown strong results in graph analysis. A graph model M can be viewed as a function that takes as input a representation of a graph $G = (X, A)$, where X (resp. A) is a matrix of node features (resp. adjacency matrix), and transforms G into a numerical matrix Z ("logits") through inference. The logits Z can then be post-processed into task-specific output for downstream analysis, such as labels for node classification. Given a test graph G and a graph model M, an "inference query" $Q(M, G)$ applies M over G to return the model output Z. Such a query may also return results $Q(M, V_T, G)$ for a set V_T of test nodes of interest in the graph.

While effective, trustworthy applications of graph models still require better methods for interpreting their outputs. Model explanation has been studied to identify subgraphs as "witnesses" that help clarify why a result appears in the inference output, by ensuring that they preserve the same or a similar model output [8]. However, erroneous or incomplete graph data may still yield witnesses that are faithful yet incomplete, inconsistent, or unconvincing. For example, molecular graph representations may contain ambiguous bond types, inconsistent charge annotations, or missing structural relations [40]. Consider the following motivating example.

Example 1: Figure 1(b) shows a concrete molecular inference scenario. A 3-layer GCN predicts the highlighted target atom v_t as

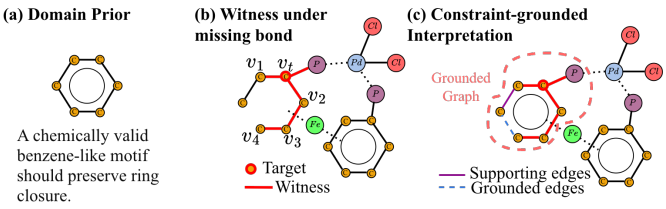


Figure 1: A witness may be semantically fragile under noisy chemical representations, while grounding recovers a more meaningful grounded graph.

mutagenic from its local 3-hop neighborhood. The red-highlighted subgraph is a witness for this prediction. It contains an open carbon chain around v_t (i.e., v_1–v_t–v_2–v_3–v_4) together with a nearby side branch, and it is sufficient for the model to return the same prediction on v_t as on the full graph. However, the bond that closes the ring is missing in the observed graph. As a result, this witness does not satisfy the domain prior in Fig. 1(a): a chemically valid benzene-like motif should preserve ring closure. □

The above example shows that a witness alone may be insufficient for interpreting inference over incomplete graphs. Even when a witness preserves the model's prediction, missing edges in the observed graph may prevent it from satisfying basic domain priors. This suggests that explanation quality should be assessed not only by faithfulness to the model output, but also by consistency with the constraints. Together, these observations motivate a method that generates witnesses that both clarify the result of an inference query and can be grounded by user-specified data constraints.

Example 2: Following Example 1, Fig. 1(c) shows how grounding augments the witness in Fig. 1(b). Specifically, one missing ring-closing edge is added as a blue dotted edge, together with one supporting edge in purple that is already present in the observed graph. The graph enclosed by the pink dashed boundary still preserves the same witness around v_t, but no longer presents it as an isolated open chain. Instead, it places the witness in a locally closed ring structure consistent with the ring-closure prior in Fig. 1(a). This example illustrates the role of grounding: rather than replacing the original witness, it augments it with one supporting edge and one missing relation, so that the resulting graph remains predictive for v_t and also satisfies the ring-closure domain prior. □

These examples highlight the need for graph inference methods that go beyond preserving the model's prediction alone. In the spirit of Why-provenance [5], we seek subgraphs that explain why a GNN inference query returns a result. Unlike conventional provenance, however, the desired subgraphs should capture not only the model-relevant part of the input, but also be consistent with external constraints. This motivates the following problem formulation.

Constraint-grounded Interpretation for Graph Inference. Given a graph G with a test node v_t, a GNN M, an inference query

$Q(M, v_t, G)$ returning the model output over v_t on G, and a constraint set Σ, the goal is to derive a grounded witness pair (G_s, G_g), where G_g is a connected grounded graph containing G_s, such that

- G_s captures a subgraph of G that helps clarify the result of $Q(M, v_t, G)$, and hence serves to assess interpretation quality relative to the model output (*e.g.,* faithfulness); and
- G_g grounds the witness G_s by satisfying one or more constraints in Σ, thereby aligning G_s with validated graph patterns that carry real-world meanings (*e.g.,* chemical compounds, social communities, or metabolic structures).

In this sense, our framework follows a bi-level view of grounded graph interpretation: G_s captures the model-relevant part of the interpretation, while G_g captures its grounding under data constraints. Hence, interpretation quality is evaluated on G_s, and grounding quality on G_g. Note that G_g need not be a subgraph of G.

Contributions. This paper makes the following contributions.

- In Section 3, we formulate *constraint-grounded interpretation for GNN inference* as a bi-level problem in which a witness graph captures the model-relevant part of the explanation and a grounded graph captures its grounding with declarative data constraints.
- Also in Section 3, we define quality measures for grounded witnesses and formulate the *top-K witness-generation problem.*
- In Section 4, we present a general framework for witness generation under constraints; Sections 5 and 6 then develop ApxIChase for general graphs and HeuIChase for tree-like neighborhoods.
- In Sections 5 and 6, we provide correctness and cost analyses, including termination, delay-time bounds for online requests, and the space cost of witness maintenance under budgets.
- In Section 7, we evaluate our methods on both real-world and synthetic graphs, and across multiple representative GNN architectures, showing that they produce high-quality, constraint-aligned witnesses with practical efficiency and substantially stronger grounded coverage than representative baselines.

To our knowledge, this is the first approach to explicitly integrate declarative graph constraints into GNN inference interpretation.

Related Work. We summarize related work below.

Provenance for Graph Queries. Provenance in data management studies how and why query results are derived from input data [9], and has primarily focused on structured queries. Recent work extends provenance to graph queries. [38] studies why-, why-not-, and why-rank questions for subgraph entity search over attributed graphs via query rewriting. [48] investigates why-questions for structural graph clustering, and [47] studies why-not questions in radius-bounded k-core search over geo-social networks. These approaches provide useful provenance analyses for symbolic graph search and mining, but they do not explain why a graph ML model, such as a GNN, produces a particular prediction. In contrast, our work builds on this line of research but focuses on grounded post-hoc interpretation for graph inference under graph data constraints.

GNN Explanation. A large body of work explains GNN predictions by identifying influential subgraphs. Representative post-hoc methods include perturbation-based methods such as GNNExplainer [43], parameterized explainers such as PGExplainer [30], surrogate-based methods such as PGMExplainer [41], and causality-inspired approaches such as GEM, RCExplainer, and OrphicX [27, 28, 42]. Recent work has also incorporated more structured reasoning, but largely stops at translating subgraphs into rules rather than using rules as explicit grounding constraints [4, 26, 44].

Overall, these methods optimize behavioral fidelity, but do not ensure that the resulting explanations satisfy user-defined or domain-specific constraints. Our approach instead evaluates a model-facing witness together with a grounded graph whose structure is constrained by declarative rules, yielding interpretations that are both faithful to the model output and aligned with domain knowledge.

2 INFERENCE, CONSTRAINTS AND WITNESS

2.1 Graph Inference Queries

Graphs. A graph $G = (V, E)$ has a set of nodes V and a set of edges $E \subseteq V \times V$. Each node v has a type $v.l$, and carries a tuple $v.T$ of attributes and their values. The size of G, denoted as $|G|$, is the total number of its edges ($|G| = |E|$). Given a node v in G, the *L-hop neighbors* of v, denoted as $N^L(v)$, are the set of all nodes within L hops of v in G. The *L-hop subgraph* of a set of nodes $V_s \subseteq V$, denoted as $G^L(V_s)$, is the subgraph induced by the node set $\bigcup_{v_s \in V_s} N^L(v_s)$.

Graph Neural Networks. GNNs [12, 24] comprise a well-established family of deep learning models tailored for analyzing graph-structured data. GNNs generally employ a multi-layer message-passing scheme as shown in Equation 1.

$$\mathbf{H}^{(l+1)} = \sigma(\widetilde{\mathbf{A}}\mathbf{H}^{(l)}\mathbf{W}^{(l)}) \tag{1}$$

$\mathbf{H}^{(l+1)}$ is the matrix of node representations at layer l, with $\mathbf{H}^{(0)} = \mathbf{X}$ being the input feature matrix. $\widetilde{\mathbf{A}}$ is a normalized adjacency matrix of an input graph G, which captures the topological feature of G. $\mathbf{W}^{(l)}$ is a learnable weight matrix at layer l (a.k.a "model weights"). σ is an activation function, such as ReLU.

The *inference process* of a GNN M with L layers takes as input a graph $G = (X, \widetilde{\mathbf{A}})$, and computes the embedding $\mathbf{H}_v^{(L)}$ for each node $v \in V$, by recursively applying the update function in Equation 1. The final layer's output $\mathbf{H}^{(L)}$ (a.k.a "output embeddings") is used to generate a task-specific *output*, by applying a post-processing layer (*e.g.,* a softmax function). We denote the task-specific output as $M(v, G)$, for the output of a GNN M at a node $v \in V$.

Deterministic Inference. A GNN M has a *fixed* inference process if its inference process is specified by fixed model parameters, number of layers, and message passing scheme. It has a *deterministic* inference process if $M(\cdot)$ generates the same result for the same input. We consider GNNs with fixed, deterministic inference processes for consistent and robust performance in practice.

Inference Query. An *inference query* Q is a triple $Q = (M, v_t, G)$ where M is a fixed, deterministic GNN model, v_t is a target node in a test graph G. The result of Q is the output $\mathbf{H}_{v_t}^{(L)}$ of M on v_t under inference over G. For fixed M and v_t, the output of Q is uniquely determined by G and a graph model M as a function $G \rightarrow \mathbf{H}_{v_t}^{(L)}$.

A query workload $\boldsymbol{Q} = (M, V_T, G)$ defined on a set of test nodes $V_T \subseteq V$ is a set of inference queries $Q(M, v_t, G)$ ($v_t \in V_T$).

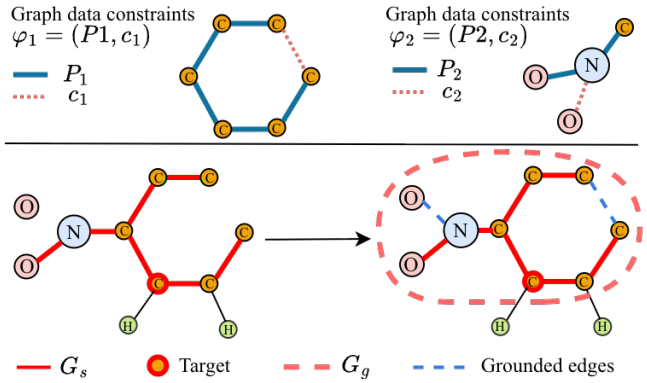


Figure 2: Illustration of two data constraints(above) and a forward grounding step(below). A witness subgraph G_s (red) matching $\varphi_1 = (P_1, c_1)$ and $\varphi_2 = (P_2, c_2)$ is extended to a grounded graph G_g (pink dashed boundary) by enforcing the missing consequents as grounded edges (blue dashed). The resulting G_g contains co-occurring matches of both antecedents and consequents, and hence satisfies both constraints.

Example 3: Consider the molecular graph illustrated in Fig. 1(b), and let M be a 3-layer GCN trained for node classification, where each node represents an atom and the task is to predict whether an atom is *mutagenic*. An inference query $Q(M, v_t, G)$ targets the highlighted atom node v_t in the graph G, asking the model to compute its final-layer representation $\mathbf{H}_{v_t}^{(3)}$ and the corresponding prediction. To answer this query, the model aggregates information from the 3-hop neighborhood of v_t and uses the resulting representation to determine whether the target atom is mutagenic. A workload $\mathcal{Q} = (M, V_T, G)$ then consists of all such per-atom inference queries for a set of target nodes V_T in the same molecular graph. □

2.2 Graph Data Constraints

We consider a class of graph data constraints (or simply, "constraints") in the form of a pair (P, c), where

(1) P ("antecedent") is a conjunction of triple patterns (a graph pattern), where each triple pattern is of the form <u_s,u_p,u_o>. Each pattern node u_s represents an entity type, and can be associated with a variable X_s and a conjunction of literals of the form u_s op a, where a is a constant, and op is a comparison operator in $\{=, >, <, \geq, \leq, \neq\}$. The predicate u_p specifies the relation between u_s and u_o, while the object u_o denotes a literal value or another typed entity.

(2) c is a single triple pattern ("consequent") in the same form as in (1), and involves at least one node (variable) seen in P.

Semantics. A constraint $\varphi = (P, c)$ declares that the antecedent pattern P and the consequent c should co-occur under the same variable assignment. We say that a graph G_g satisfies φ, denoted by $G_g \models \varphi$, if there exists a homomorphism h such that $h(P) \subseteq G_g$ and $h(c) \in E(G_g)$. For a graph G and a target node v, we say that $(G, v) \models \varphi$ if there exists such a graph G_g containing v. For a fixed target node, we simply write $G \models \varphi$. In our setting, G_g serves as grounded graph associated with a witness G_s for an inference query.

In practice, such constraints can encode *Type*: that entities must have a certain type (*e.g.*, verifying that *"a person is identified as an actor or director"* in a movie-related query); *Value Binding*: Restricting values for literals (*e.g.*, ensuring that a date falls within a specific range or that a number must be above a threshold); or *Structural Constraints*: Declaring relationships with cardinality constraints (*e.g.,a director must have directed at least one film*").

Given a set of constraints Σ, the graph G satisfies Σ, denoted as $G \models \Sigma$, if for every constraint $\varphi \in \Sigma$, $G \models \varphi$. We say a graph data constraint φ is *trivial*, if $c \in P$. Given a pair of graph data constraints (φ, φ'), we say $\varphi = (P, c)$ implies $\varphi' = (P', c)$, if $P \subseteq P'$. In our work, we assume a set of non-trivial graph data constraints.

Remarks. In this work, we focus on witness grounding under a given set of predefined constraints; the problem of constraint discovery or mining is orthogonal to our study. In practice, such constraints may come from several established sources: (1) rule mining, via automated discovery of graph association rules or dependencies [15]; (2) public schemas and ontologies, such as domain-specific knowledge bases (e.g., UMLS for medicine) or RDF shapes (SHACL) [35]; and (3) expert knowledge, via manually specified semantic requirements in domains such as chemistry or finance.

Enforcement. Chase [6] has long been a powerful tool for query expressiveness, constraint reasoning and enforcement. Given a set of data constraints Σ and a dataset D, a Chase process is a general technique that can transform D such that $D \models \Sigma$. When Σ is used to express a query Q (hence a set of query constraints), Chase can be used for evaluating conjunctive queries. If Σ is defined as data quality rules or constraints [31], Chase enforces Σ as a data repairing process. Applying Chase twice yields Chase&BackChase, a classic technique used for query optimization or query suggestion [11]. In this work, we borrow Chase mainly as a local grounding mechanism for constructing grounded graphs around a witness, rather than as a global repair process over the original graph.

Example 4: Consider the witness subgraph G_s in Fig. 2, and two constraints $\varphi_1 = (P_1, c_1)$ and $\varphi_2 = (P_2, c_2)$. Here P_1 specifies a five-edge carbon chain, and c_1 requires the closing bond between its two endpoints, forming a six-carbon ring: $P_1 = \{\langle C_i, bond, C_{i+1}\rangle \mid 1 \leq i \leq 5\}, c_1 = \langle C_6, bond, C_1\rangle$. Moreover, P_2 requires that a nitrogen atom be bonded to an oxygen atom and a carbon atom, while c_2 adds another nitrogen–oxygen bond: $P_2 = \{\langle N, bond, O_1\rangle, \langle N, bond, C\rangle\}, c_2 = \langle N, bond, O_2\rangle$. In Fig. 2, the red subgraph G_s matches antecedents P_1 and P_2. A grounding step then enforces the two consequents by introducing the missing relations as blue dashed edges, yielding the graph G_g on the right. As a result, G_g satisfies both constraints, *i.e.*, $G_g \models \{\varphi_1, \varphi_2\}$. □

2.3 Witnesses for Graph Inference Queries

To characterize the subgraphs used to interpret graph inference queries, we introduce the notion of witness graphs.

Witness Graph. Given an inference query $Q = (M, v_t, G)$ over a test node v_t and a graph ML model $M : G \rightarrow \mathbf{H}_v^{(L)}$, a subgraph G_s of G is a *witness* of the result of Q if G_s contains v_t and satisfies *one* of the following two conditions:

- $Q(M, v_t, G) = Q(M, v_t, G_s)$ ("factual"); or
- $Q(M, v_t, G) \neq Q(M, v_t, G \setminus G_s)$ ("counterfactual")

Example 5: Given the inference query $Q(M, v_t, G)$ in Example 3, the red subgraph shown in Fig. 1(b) serves as a witness for the prediction on v_t. It captures the local carbon-chain fragment containing v_t, together with its linkage to a phosphorus atom, from which the 3-layer GCN M derives the representation $\mathbf{H}_{v_t}^{(3)}$ and the corresponding mutagenicity prediction. In this sense, the witness isolates the local structure used by the model to produce the prediction. □

A witness for graph inference, in analogy to Why-provenance [5], refers to a graph that is necessary to reconstruct the inference result of $Q(M, v_t, G)$ (as a factual witness), or to change the latter if removed (as a counterfactual witness). A witness can be explicitly generated by a post-hoc "explainer" that ensures a factual witness [43] or a counterfactual witness [29], or be implicitly induced from a learned masked adjacency matrix representation [30].

3 GROUNDING WITNESSES WITH CONSTRAINTS

As remarked earlier, a witness alone may not be aligned to meaningful real-world structures as it may only be faithful to the model [43]. We next describe how data constraints can be exploited to "ground" witnesses. We start by introducing a notion of k-grounded witness.

k-Grounded Witness. Given a data constraint φ, a witness G_s of the result of $Q(M, v_t, G)$, and a number k, we say G_s is *k-grounded* by φ, denoted by $G_s \models_k \varphi$, if there exists a corresponding grounded graph $G_g = G_s \oplus \Delta E$, *i.e.,* obtained by adding at most k new edges ΔE that are not in G, such that (1) $G_s \subseteq G_g$, and (2) $G_g \models \varphi$ ($|\Delta E| \leq k$). We refer to the edges in ΔE as *grounded edges*.

We remark that the symbol $\oplus$ means "extended" but not "union", *i.e.,* G_g may contain G_s, a set of new edges ΔE not in G, and a set of edges in G but not in G_s – to make G_g a cohesive, connected graph.

A witness G_s is k-grounded by any subset of data constraints $\widehat{\Sigma} \subseteq \Sigma$, if for every constraint $\varphi \in \widehat{\Sigma}$, $G_s \models_k \varphi$. When $\widehat{\Sigma} = \Sigma$, this corresponds to a fully grounded witness. Ideally, G_s is 0-grounded by Σ, *i.e.,* $G_s \models_0 \Sigma$. Intuitively, if a witness G_s is k-grounded (by φ or by $\widehat{\Sigma}$), then G_s is also $(k+1)$-grounded, since the same grounded graph remains feasible under the larger edge-insertion budget. A k-grounded witness G_s is *minimal* for a given φ if (1) G_s is a minimal witness, and (2) ΔE is a minimal set of edges that ensures $G_s \models_k \varphi$.

Example 6: Consider $\Sigma = \{\varphi_1, \varphi_2\}$ in Fig. 2, where φ_1 enforces the carbon ring closure and φ_2 regulates the nitrogen–oxygen bonding pattern. Let G_s be the witness (red bold edges) that matches P_1 and P_2, and let $G_g = G_s \oplus \Delta E$ be the extended graph enclosed by the pink dotted boundary. Here ΔE denotes the blue dashed links that do not appear in the original G, which correspond to the consequents c_1 and c_2, respectively. Specifically, the added edge (C_6, C_1) enables $G_g \models \varphi_1$ by closing the carbon ring, and (N, O_2) ensures $G_g \models \varphi_2$ by completing the nitrogen–oxygen linkage. Hence $G_s \models_1 \varphi_1$ and $G_s \models_1 \varphi_2$, indicating that G_s is 1-grounded by Σ. □

Bi-level Quality Measures. As there are many witnesses that can be grounded under constraints, we next assess their quality from two complementary aspects: the quality of each individual witness, and the collective quality of a selected witness set.

Conciseness & Minimality. A witness should be concise, following Occam's Razor, by retaining as few edges as possible. (1) A witness G_s is *minimal* for an inference query $Q(M, v_t, G)$, if any of its subgraph obtained by removing an edge is not a witness of $Q(M, v_t, G)$. (2) Consider $G^L(v_t)$, the L-hop subgraph of a test node v_t. The conciseness score of a witness G_s is defined as

$$\mathsf{conc}(G_s) = 1 - \frac{|G_s|}{|G^L(v_t)|}.$$

We aim to find minimal witnesses with higher conc scores.

Alignment. For a witness G_s and its corresponding grounded graph G_g, let $\Sigma'(G_g) \subseteq \Sigma$ be the subset of constraints grounded by G_g. Since k is a local budget for each constraint in Σ', grounding them independently would require at most $k \cdot |\Sigma'|$ new edges. We therefore define the alignment score of G_g with respect to Σ' as

$$\mathsf{aln}(G_g) = 1 - \frac{|\Delta E|}{k \cdot |\Sigma'(G_g)|},$$

when $|\Sigma'(G_g)| > 0$, and $\mathsf{aln}(G_g) = 0$ otherwise. Here a higher aln score indicates that G_s can be grounded with fewer added edges, relative to the total grounding budget allowed by Σ'. In particular, if no new additional edge is needed, then $\mathsf{aln}(G_g) = 1$.

Per-witness Quality. We first define the quality of a pair (G_s, G_g) as

$$q(G_s, G_g) = \alpha \cdot \mathsf{conc}(G_s) + \beta \cdot \mathsf{aln}(G_g),$$

where $\alpha, \beta \geq 0$ are tunable weights. This score independently captures the local quality of an individual grounded witness pair.

Coverage on Witness Sets. Coverage is inherently a set-level notion, since different witnesses may ground different subsets of constraints. For a set of selected grounded witness pairs $\mathcal{P} = \{(G_s^1, G_g^1), \ldots, (G_s^K, G_g^K)\}$, we define its covered constraint set as $\mathsf{CovSet}(\mathcal{P}) = \bigcup_{(G_s, G_g) \in \mathcal{P}} \Sigma'(G_g)$, where $\Sigma'(G_g) \subseteq \Sigma$ is the set of constraints grounded by G_g. We define a set-level coverage score as

$$\mathsf{cov}(\mathcal{P}) = \frac{|\mathsf{CovSet}(\mathcal{P})|}{|\Sigma|}.$$

A higher $\mathsf{cov}(\mathcal{P})$ indicates that the selected witness set collectively covers a larger portion of the constraint set, thus providing more comprehensive grounding for the selected witness set as a whole.

Overall Quality Objective. We favor a set of grounded witness pairs whose individual members are concise and well aligned, while collectively covering as many data constraints as possible. Therefore, we define the overall quality score of a witness pair set $\mathcal{P}$ as

$$F(\mathcal{P}) = \sum_{(G_s, G_g) \in \mathcal{P}} q(G_s, G_g) + \gamma \cdot \mathsf{cov}(\mathcal{P}),$$

where $\gamma \geq 0$ is a tunable weight on the set-level coverage term.

An Axiomatic Analysis. We justify the set objective $F(\mathcal{P})$ by the following properties. For any inference query $Q(M, v_t, G)$, let $\mathcal{P}^*$ be an optimal witness set that maximizes $F(\mathcal{P})$ under the cardinality constraint $|\mathcal{P}| \leq K$. The function $F(\mathcal{P})$ has the following properties.

(1) **Scale invariance**. $\mathcal{P}^*$ remains optimal if the component scores in $q(G_s, G_g)$ and the coverage term $\mathsf{cov}(\mathcal{P})$ are scaled by a common constant, provided that the corresponding weights are scaled accordingly. This ensures that the optimal witness set is invariant to the normalization of user preference weights.

(2) **Non-monotonicity**. $F(\mathcal{P})$ is not necessarily monotone with respect to the sizes of individual witnesses or grounded graphs. Indeed, selecting larger witnesses does not necessarily improve the overall quality, since additional edges may reduce conciseness without proportionally improving alignment or coverage.

(3) **Locality.** The objective $F(\mathcal{P})$ respects a locality principle. Specifically, the local quality of each selected pair $(G_s, G_g) \in \mathcal{P}$ is captured by $q(G_s, G_g)$, while interactions among selected pairs arise only through the set-level coverage term $\text{cov}(\mathcal{P})$. No information outside the selected witness set and their corresponding grounded graphs affects the overall quality evaluation here. This reflects a pragmatic semi-closed-world view: we evaluate grounded interpretations with respect to the observed graph and a bounded set of grounded edges, rather than reasoning over arbitrary unseen facts or assuming that the observed graph is already complete.

Problem Statement. Given an inference query $Q(M, v_t, G)$, a budget k, a set of data constraints Σ, and an integer K, the problem of *Top-K witness generation* computes a K-set of grounded witness pairs $\mathcal{P} = \{(G_s^1, G_g^1), \ldots, (G_s^K, G_g^K)\}$, such that (1) each pair $(G_s, G_g) \in \mathcal{P}$ consists of a minimal k-grounded witness G_s and a corresponding grounded graph G_g, where G_g grounds G_s under a subset $\Sigma'(G_g) \subseteq \Sigma$ of one or more constraints; and (2) $\mathcal{P}$ maximizes the overall set objective:

$$\mathcal{P}^* = \arg \max_{\substack{\mathcal{P}' \subseteq C \\ |\mathcal{P}'| \leq K}} F(\mathcal{P}'),$$

where C denotes the set of feasible candidate grounded witness pairs. This problem is non-trivial as verified by the hardness result below (see detailed hardness analysis in [2]). A witness G_s may admit multiple grounded graphs G_g, since different constraints in Σ may justify the same witness under the budget k, and the same grounded graph G_g may in turn support different witnesses. Note that grounding does not physically repair the original graph G; each G_g is derived independently as an auxiliary structure.

Decision Version. We consider the associated decision version of top-K witness generation: given an inference query $Q(M, v_t, G)$, a budget k, a set of constraints Σ, and an integer K, determine whether there exists a grounded witness set $\mathcal{P}$ with $|\mathcal{P}| \leq K$.

Theorem 1: *The decision version of the witness generation is NP-hard, when GNN model M is fixed, $K = 1$, $k = 0$, and $|\Sigma| = 1$.* □

Proof sketch: By reduction from *Clique*: we fix the GNN so that witness existence is trivial, and encode clique existence by a single grounding constraint, so a feasible grounded witness pair exists if and only if the input graph contains the required clique. Full details are deferred to the appendix/full version [2]. □

We next present practical algorithms to generate grounded witness pairs for graph inference under constraints.

A naive solution. We start with a simple algorithm, denoted as naiveChase, that combines fixed-point enforcement, graph pattern matching, and witness verification to generate k-grounded witnesses. The procedure follows an "enforce-and-ground" process. (1) Given a query $Q(M, v_t, G)$ and a constraint set Σ, naiveChase first enforces Σ on the local input graph $G^L(v_t)$ via a fixed-point computation. This is done by a variant of the Chase algorithm, denoted as I-Chase, which iteratively inserts an edge matching a consequent $c(u, u')$ whenever a subgraph match of the corresponding antecedent P is found, and terminates when no constraint can be further enforced. The lemma below guarantees termination and yields a unique chase-saturated graph G' up to graph isomorphism, in which no further applicable constraint can be enforced.

Lemma 2: *Given a graph G and a set of constraints Σ, (1)* I-Chase *always terminates after finitely many enforcement steps, and (2) it returns a unique chase-saturated result up to graph isomorphism.* □

(2) For each $\varphi = (P, c)$ and each subgraph G_s that matches P, naiveChase invokes the verification procedure VerifyWitness (Section 3) to check whether G_s is a witness for $Q(M, v_t, G)$. For each verified witness G_s, it then derives candidate grounded graphs G_g from the chased graph G'. Since G' is chase-saturated with respect to the applicable antecedent matches in the local graph, any verified witness G_s that contains a triggered match of P admits a corresponding grounded graph G_g with $\Delta E = 0$ for φ in G'.

While naiveChase can derive a set of grounded witnesses (see Analysis in [2]), it remains infeasible in practice for large G with missing edges. (1) Enforcing Σ is inherently expensive, due to an exhaustive enumeration of subgraphs that match P by subgraph matching, which is already computationally hard and costly in practice. (2) The resulting grounded pairs (G_s, G_g) are extracted from a refined graph G' that may contain many enforced edges not present in the original input graph G, weakening their connection to the observed input. (3) The extracted grounded graphs may contain redundant structure beyond what is needed for a match of $P \cup c$.

We next introduce efficient algorithms, to generate k-grounded witnesses without an exhaustive enforcement of Σ over input graph.

4 WITNESS GENERATION: OVERVIEW

We introduce a general algorithm, denoted as Inf-Chase.

Outline. The foundation for our algorithm is a Chase&Backchase-style grounding framework. It undergoes two phases below:

(1) A **Chase** phase, which enforces constraints from Σ over the L-hop subgraph $G^L(v_t)$ of v_t whenever they are applicable, via a procedure **L-Chase**, a variant of I-Chase that performs bounded rounds of reasoning (see "L-Chase"); and

(2) A **Backchase** phase, which uses an ***inverse*** set of constraints derived from Σ to ground each witness in $\mathcal{G}_s$ under budget k. This is supported by a procedure **L-Backchase**, which uses an input inverse constraint set $\Sigma_b \subseteq \Sigma^{-1}$ on G_s, together with an edge update budget k, to obtain a graph G_g that grounds the witness G_s. Here each data constraint $\varphi^{-1} = (c, P)$ in Σ_b is an inverse counterpart of some $\varphi = (P, c)$ in Σ. Putting together, the process derives a K-set of grounded witness pairs $\{(G_s^1, G_g^1), \ldots, (G_s^K, G_g^K)\}$, where $\mathcal{G}_s = \{G_s^1, \ldots, G_s^K\}$ and $\mathcal{G}_g = \{G_g^1, \ldots, G_g^K\}$ denote the corresponding witness set and grounded-graph set, respectively. For each pair (G_s^i, G_g^i), the graph G_g^i contains a k-grounded witness G_s^i, and satisfies one or more inverse constraints in Σ_b. The latter holds due to the co-occurrence of matches of both c and P in G_g^i. The overall computation essentially repeats the two steps below up to L times.

$$\mathcal{G}_s := \text{I-Chase}\ (\Sigma, G^L(v_t), 0); \quad \mathcal{G}_g := \text{I-Chase}\ (\Sigma_b, \mathcal{G}_s, k).$$

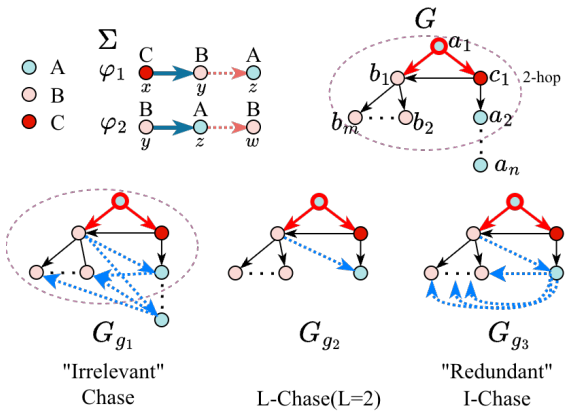


Figure 3: Generating grounded witnesses (Inf-Chase); unlike I-Chase as a fixpoint computation, L-Chase reasons with Σ up to L times from a set of "source" constraints Σ_s.

Procedure L-Chase. Unlike I-Chase, L-Chase differs in the following. (1) Rather than a fixpoint computation as in I-Chase, it performs up to L rounds of Chase, starting from v_t and a set of "source" constraints Σ_s with pattern nodes having v_t as a match, to avoid "over grounding" the output $Q(M, v_t, G)$ with real-world entities that are too far or irrelevant to v_t. (2) L-Chase directly derives a set $\mathcal{G}_s$ of verified witnesses that are already 0-grounded in the local input graph. Specifically, for each $\varphi = (P, c) \in \Sigma$, L-Chase ($\Sigma$, G, k) follows a graph pattern mining process to generate a set of candidate subgraphs $\mathcal{G}_c$ via an edge growing strategy, and invokes procedure VerifyWitness to retain those that are witnesses.

Procedure L-Backchase. If $k > 0$ ("Backchase"), then procedure L-Backchase finds all $\varphi^{-1} = (c, P)$ with consequent c having an edge match e_c in G_s, and inserts a minimal set ΔE of at most k new edges to e_c, to enforce at least one occurrence of a match of P. As such, G_s is k-grounded by φ^{-1} in this backchase step. Including a connected subgraph G_g with G_s and ΔE then ensures the generation of a set of minimal grounded graphs $\mathcal{G}_g$ for the witness. This step introduces at most $|\Sigma|k$ new grounded edges in total.

Example 7: Consider the constraint set $\Sigma = \{\varphi_1, \varphi_2\}$ shown in Fig. 3. All nodes a_i in G have type A; similarly, nodes b_i and c_i have types B and C, respectively. Constraint $\varphi_1 = (P_1, c_1)$ states that whenever a node of type B has an incoming edge from a node of type C, it should also have an outgoing edge to a node of type A. Constraint $\varphi_2 = (P_2, c_2)$ states that whenever a pattern $B \rightarrow A$ is present, a corresponding edge $A \rightarrow B$ should also exist. On the right-hand side of Fig. 3, the red bold edges denote a witness G_s rooted at the target node v_t, and the dashed oval marks its 2-hop neighborhood $G^2(v_t)$. We compare three outcomes.

(1) As P_1 is matched, a Chase, as a fixpoint process, enforces c_1 to insert edges between b_2 and any a node regardless of how large n is ($b_2 \rightarrow a_2, \ldots, a_n$). Most of these edges fall outside of the receptive field of v_t and thus become irrelevant (see G_{g_1}).

(2) In contrast, I-Chase restricts reasoning to $G^2(v_t)$, so nodes outside the local neighborhood are excluded. However, after φ_2 is triggered, it may still generate multiple semantically equivalent local structures (e.g., $a_2 \rightarrow b_2, \ldots, b_m$), leading to an excessive number of over-grounded graphs that are mutually isomorphic (see G_{g_3}).

(3) L-Chase strikes a balanced alternative: since $L = 2$ and the witness G_s already covers the hop-1 neighborhood of v_t, one additional round of local reasoning within $G^L(v_t)$ is enough to produce the grounded result G_{g_2}. This avoids both the irrelevant edges of G_{g_1} and the redundant local structures of G_{g_3}. The example thus illustrates why bounded hop ranges and bounded reasoning rounds are preferable in practice for witness grounding. □

Correctness. We show that Inf-Chase correctly computes a set of grounded graphs $\mathcal{G}_g$ such that each $G_g \in \mathcal{G}_g$ contains a k-grounded witness G_s under some constraint $\varphi \in \Sigma$. We observe two invariants hold throughout the execution of Inf-Chase.

(1) The Chase phase identifies a set of subgraphs $\mathcal{G}_s$ of $G^L(v_t)$ such that, for each $G_s \in \mathcal{G}_s$, G_s already contains a co-occurring match of P and c for some $\varphi = (P, c) \in \Sigma$, and is therefore 0-grounded; and G_s has been verified by VerifyWitness to be either a factual or a counterfactual witness for $Q(M, v_t, G)$.

(2) For each witness $G_s \in \mathcal{G}_s$ derived in the Chase phase, there exists some constraint $\varphi = (P, c) \in \Sigma$ such that G_s already contains a match of the consequent c. The backchase phase then considers the inverse form $\varphi^{-1} = (c, P)$ and applies I−Chase(φ^{-1}, G_s, k) to extend G_s under a local budget of at most k grounded edges for that match instance. If successful, the resulting graph G_g contains both the original match of c and a recovered match of P, and therefore contains G_s as a k-grounded witness under the corresponding constraint in Σ. Repeating this process for all applicable inverse constraints over all witnesses $G_s \in \mathcal{G}_s$ yields the grounded graphs returned by Inf-Chase. In this sense, Inf-Chase is a two-phase Chase&Backchase process: it first derives witnesses in $G^L(v_t)$, and then grounds them by applying inverse constraints under the budget k.

Time Cost. Both Chase and backchase phases takes $|\Sigma|$ calls of I-Chase, a fixed-point reasoning process as a sequence of L enforcement of constraints. It takes in total $O(|G^L(v_t)|^p)$ time to find matches of patterns from Σ, where p is the size of the largest pattern P a data constraint has in Σ. The inference cost is in $O(L|G^L(v_t)||N^L(v_t)|)$ time [7, 45], assuming $N^L(v_t)$ is much larger than the number of node features. Taken together, this yields a cost of Inf-Chase in $O(|G^L(v_t)|^p) + L|\Sigma||G^L(v_t)||N^L(v_t)|$.

While Inf-Chase can generate grounded witness pairs, (1) the resulting set may not optimize the overall quality objective; and (2) the exhaustive exploration of subgraphs in $G^L(v_t)$ remains expensive in practice. We next introduce two practical algorithms to make Inf-Chase feasible for large scale graphs.

5 ONLINE WITNESSES GENERATION

Our first algorithm, denoted as ApxIChase and illustrated in Fig. 4, instantiates Inf-Chase as an online generation process with (1) a further optimization of L-Chase with cost-effective and memoization structures, to reduce excessive inference and verification cost; and (2) ensures to return top-K k-grounded witness pairs with an "any-time" quality guarantee under a small request-time delay.

Algorithm 1 ApxIChase

Input: Constraint set Σ, query $Q(M, G, v_t)$, integers L, K, and k;
Output: the current top-K grounded witness pair set W_K.

1: construct $G_\Sigma := \bigcup_{\varphi \in \Sigma}(P \cup c)$; initialize $\mathcal{M}$;
2: set $\mathcal{G}_s := \emptyset$, $C := \emptyset$, $W_K := \emptyset$;
3: **for** $\ell = 1$ **to** L **do**
4: $\quad \Sigma_s := \{\varphi \mid u.l = v_t.l,\ u$ is from $\varphi.P,\ \varphi \in \Sigma\}$ /* Chase phase */
5: $\quad \mathcal{G}_s := \mathcal{G}_s \cup \mathsf{L-Chase}(\Sigma, \Sigma_s, G^L(v_t), G_\Sigma, \mathcal{M}, 0)$
6: $\quad$ **for each** $G_s \in \mathcal{G}_s$ **do**
7: $\quad\quad \Sigma_b := \emptyset$
8: $\quad\quad$ **for each** $\varphi \in \Sigma$ **do**
9: $\quad\quad\quad$ **if** $G_s \models \varphi$ **then** $\Sigma_b := \Sigma_b \cup \varphi^{-1}$
10: $\quad\quad \mathcal{G}_g := \mathsf{L-Backchase}(\Sigma_b, G_s, G_\Sigma, \mathcal{M}, k)$ /* Backchase phase */
11: $\quad\quad C := C \cup \{(G_s, G_g) \mid G_g \in \mathcal{G}_g\}$
12: $\quad\quad W_K := \mathsf{UpdateWK}(C, K)$
13: **return** W_K

Figure 4: Algorithm ApxIChase**: Chase & Backchase with bounded enforcement and edge insertion.**

Theorem 3: *There is an online algorithm that returns a top-K set of k-grounded witness pairs, upon request time, with a delay time in $O(|\Sigma|^2|G^L(v_t)| + L|\Sigma|^2 + |\Sigma||C|K)$, and ensures a $(1 - 1/e)$-approximation ratio to the current optimum of $F(\mathcal{P})$ obtainable from the grounded witness pairs generated so far.* □

C in theorem 3 denotes the current candidate pool of grounded witness pairs maintained by ApxIChase.

Memoization Structures. ApxIChase stores and maintains the current top-K grounded witness pair set W_K together with a candidate pair pool C. Besides, it also keeps track of two memoization structures below. (1) G_Σ is a graph pattern obtained as the union of the patterns $P \cup c$, encoding all the triple patterns from data constraints of Σ. Each edge $e = (u_s, u_p, u_o)$ in G_Σ carries a set of labels $\mathcal{L}(e) = \{i | \text{if } e \text{ is in } P_i \text{ or } c_i\}$ for any $\varphi_i = (P_i, c_i)$, *i.e.,* e indicates a common triple pattern in multiple constraints.

(2) A shared map $\mathcal{M}$ with size $|\Sigma| \times (p + 2)$, with p the size of the largest pattern P in Σ, and for each entry $\mathcal{M}[i]$:

- $\mathcal{M}[i].\mathcal{P}$ is a size-$p + 1$ binary vector, such that each slot $\mathcal{M}[i].\mathcal{P}[j]$ ($j \in [1, p]$) = 1 (resp. $\mathcal{M}[i].\mathcal{P}[p + 1] = 1$) if and only if φ_i has its j-th triple pattern in P_i (resp. $\varphi_i.c_i$) has a non-empty edge match, in a current candidate subgraph G_s under verification; and
- $\mathcal{M}[i].b$, a budget number that records a lower bound of edges required to make G_s k-grounded by φ_i.

In addition, ApxIChase keeps a set of source constraints $\Sigma_s \subseteq \Sigma$ to start the enforcement in Chase phase; and for each witness G_s it derives a inverse constraint set $\Sigma_b \subseteq \Sigma^{-1}$ for the backchase phase.

Algorithm Outline. The main driving algorithm ApxIChase and the procedure L-Chase, L-Backchase are illustrated in Figs. 4 and 5, respectively; The detailed pseudocode of L-Backchase is deferred to [2]. ApxIChase first initializes the auxiliary structures above (lines 1–2). It then performs up to L rounds (line 3) of L-Chase (line 5) and, for each witness produced in the current round, invokes L-Backchase to construct the corresponding grounded graphs (line 10), consistently following Inf-Chase. The resulting grounded witness pairs are accumulated into the candidate pool C and then passed to UpdateWK, which, upon request time, greedily refreshes the current top-K grounded witness pairs under the set objective $F(\mathcal{P})$, thereby achieving an approximation ratio of $1 - \frac{1}{e}$ for the optimal K-set available at request time (see details of UpdateWK in [2]). The correctness and time cost follow from the counterpart of Inf-Chase; we present the detailed analysis in [2].

Algorithm 2 Procedure L-Chase $(\Sigma, \Sigma_s, G^L(v_t), G_\Sigma, \mathcal{M}, 0)$

1: set $\mathcal{G}_c := \emptyset$; $\mathcal{G}_s := \emptyset$;
2: **for** $\varphi \in \Sigma_s$ **do**
3: $\quad$ graph $G_c :=$ getNext$(\varphi, G_\Sigma, \mathcal{M})$;
4: $\quad$ **if** $G_c \neq \emptyset$ **then**
5: $\quad\quad \mathcal{G}_c := \mathcal{G}_c \cup \{G_c\}$; update G_Σ, $\mathcal{M}$;
6: $\quad\quad$ **if** VerifyWitness$(G_c, Q(M, v_t, G)) =$ true **then**
7: $\quad\quad\quad \mathcal{G}_s := \mathcal{G}_s \cup \{G_c\}$;
8: **return** $\mathcal{G}_s$;

Figure 5: Procedure L-Chase

Optimization. Algorithm ApxIChase uses several optimization strategies with different focuses.

(1) To avoid irrelevant grounding that contributes little to $Q(M, v_t, G)$, procedure L-Chase calls a function getNext (line 3), which follows a prioritized traversal of G_Σ to prefer constraints whose pattern edges occur more frequently in Σ, and grows candidate subgraphs G_c accordingly. This favors common constraint patterns and reduces the exploration of rare or less representative ones. Other query selectivity or cardinality estimation techniques, e.g., [21, 36], are also readily applicable here.

(2) To reduce redundant inference cost, procedure VerifyWitness adopts an incremental inference strategy. Instead of restarting the inference process from scratch for every candidate (line 6 of VerifyWitness), it first performs a once-for-all inference for $Q(M, v_t, G^L(v_t))$ and caches intermediate node embeddings. When verifying a candidate G_c, it reuses cached results whenever possible and incrementally updates only the portion of message passing affected by G_c, thereby reducing repeated computation.

(3) Procedure L-Backchase reduces grounding cost by monitoring $\mathcal{M}$. In particular, it maintains a lower bound on $|\Delta E|$ as the number of missing edge matches, recorded by the number of 0 entries in $\mathcal{M}[i].P$. The map $\mathcal{M}$ also enables early detection and run-time pruning for Σ_s and Σ_b by checking whether $\mathcal{M}[i].P[d + 1]$ (the consequent) already has a match.

(4) ApxIChase also copes with large constraint sets Σ by pruning constraints that are logically implied by others. We present the details in the appendix/full version [2].

These strategies are effective: as verified in our experimental study, they improve the naive implementation naiveChase by one to two orders of magnitude in efficiency on average.

Example 8: Consider running ApxIChase on the example in Fig. 6. The original graph G (top-left) contains six nodes a, b, c, d, e, f, whose types are A, B, C, D, E, F, respectively, with four constraints $\Sigma = \{\varphi_0, \varphi_1, \varphi_2, \varphi_3\}$ shown on the left. Each constraint $\varphi_i = (P_i, c_i)$ is

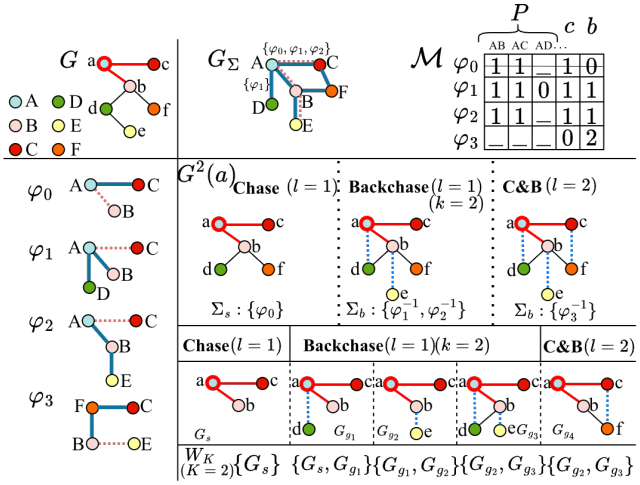


Figure 6: ApxIChase**: running of C& B in** 2 **rounds**

illustrated with its antecedent P_i (blue solid edges) and consequent c_i (pink dotted edge). The merged pattern G_Σ (top-right) encodes the union of $P_i \cup c_i$ for all $\varphi_i \in \Sigma$. The memoization map $\mathcal{M}$ records the grounding status of each constraint after one Chase round. For example, $\mathcal{M}[0].b = 0$ (as all the pattern edges have a match without "0" entries) indicates that $G_s \models_0 \varphi_0$, hence φ_0 is satisfied and can be removed from subsequent enforcement.

(1) During the first Chase phase, the receptive field of the target node a (*i.e.*, $G^2(a)$) has $\Sigma_s = \{\varphi_0\}$. In the first Backchase, given budget $k = 2$, the inverse constraint set $\Sigma_b = \{\varphi_1^{-1}, \varphi_2^{-1}\}$ is enforced.

(2) In the second round ($L = 2$), constraint φ_3^{-1} is also triggered, enforcing the new edge (c, f). The bottom row shows the evolution of the top-K window W_K (with $K = 2$), whose entries are grounded witness pairs; for visual simplicity, we display only their corresponding grounded graphs. Initially, G_s is discovered as a 0-grounded witness for φ_0. G_{g_1} and G_{g_2} represent $G_s \models_1 \varphi_1$ and $G_s \models_1 \varphi_2$, updating $W_K = \{G_{g_1}, G_{g_2}\}$. Upon the arrival of G_{g_3}, ApxIChase prefers G_{g_3} given its better coverage of both φ_1 and φ_2, and replaces G_{g_1}. The current top-K set becomes $\{G_{g_2}, G_{g_3}\}$.

(3) The process continues, and after another round of Chase & Backchase, a new graph G_{g_4} satisfies φ_3 ($G_s \models_1 \varphi_3$), yet due to a weak link to a, it does not improve the F-score ranking, hence W_K remains unchanged. The algorithm returns W_K as the result. □

6 FAST WITNESSES GENERATION

Our second algorithm specifies Inf-Chase for large, sparse, hierarchical, and tree-like graphs, such as network traffic graphs, power delivery networks, epidemic networks, or citation networks [33]. In such scenarios, the grounded graphs are often close to a tree.

Grounding Witnesses as Trees. We consider a practical variant of our problem, where (1) each constraint $\varphi = (P, c)$ in Σ has a tree pattern $P \cup c$, (2) each witness $G_s \in \mathcal{G}_s$ is a weighted tree rooted at v_t, and (3) each grounded graph $G_g \in \mathcal{G}_g$ is a weighted arborescence of $G^L(v_t)$ rooted at v_t and containing G_s. This setting aligns G_g with the message-passing structure of the inference query $Q(M, v_t, G)$. A weight function $w(e)$ for an edge $(u, w) \in G$ can be introduced using representative edge scores adopted in GNN behavior and explainability analysis [30], such as embedding similarity, where $w(e) = \text{sim}(h_u^{(L)}, h_w^{(L)}) + \varepsilon_e$, $\text{sim}(a, b) = \frac{a^\top b}{\|a\| \, \|b\|}$, and ε_e is a small random noise (*e.g.*, Gumbel or Gaussian) that introduces diversity across m sampled trees. Other options such as resistance distance [46], Shapley value [44], or sensitivity [25] can also be applied. The problem remains computationally challenging in general due to the cost of witness discovery by graph pattern matching. We next introduce a fast heuristic, denoted as HeuIChase, to generate grounded graphs G_g as maximum weighted arborescences.

Outline. The algorithm follows the framework Inf-Chase with a L-Chase step and a L-Backchase step to generate grounded witnesses. Unlike ApxIChase, HeuIChase makes the following specifications.

(1) It uses a revised L-Chase, which generates witnesses G_s via tree pattern matching and verification between a tree pattern P and a set of m sampled trees from $G^L(v_t)$. (2) It extends each G_s to a weighted arborescence G_g rooted at v_t, which serves as a heuristic realization of L-Backchase. To ensure G_s is contained in G_g, it shrinks G_s to a "super node" and uses the contracted graph of $G^L(v_t)$ as input. This reduces the problem to computing a maximum-weight arborescence. Edmonds' algorithm [13] is then applied to derive G_g over the contracted graph. The procedure UpdateWK remains unchanged and maintains the top-K grounded witness pairs greedily.

Analysis. The algorithm HeuIChase correctly generates feasible grounded witness pairs in the tree/arborescence setting, following the framework of Inf-Chase and the correctness of Edmonds' algorithm. Its main cost saving comes from replacing general subgraph enumeration by tree matching and replacing explicit backchase repair by arborescence expansion. Treating the maximum tree-pattern size and the pruned rule-pool size as small constants, the cost up to request time can be summarized as $O(Lm|G^L(v_t)| + |C||G^L(v_t)| \log |G^L(v_t)| + |C|K)$. For HeuIChase, we set $\beta = 0$ and omit the explicit alignment term in the pairwise score used by UpdateWK. See [2] for the detailed derivation.

7 EXPERIMENTAL STUDY

7.1 Experimental Settings

Using real-world and synthetic benchmark datasets, we evaluate the effectiveness, efficiency, and scalability of our algorithms.

Datasets. We use five real-world graphs and two synthetic graphs for evaluation (summarized in Table 1). (1) *MUTAG* [10], a set of 188 chemical compound graphs. Each graph has a class label indicating whether it is mutagenic or non-mutagenic. Node feature is a 7-dimensional one-hot encoding of atom types (*e.g.*, C, N, O). (2) *ATLAS* [3] is a cyber event graph for intrusion detection. Nodes represent system entities (e.g., process, file, connection), and edges denote data flow or dependencies. (3) *Cora* [33] is a citation network with 7 categories. Each node has a 1,433-dimensional feature vector, and edges represent citation links among papers. (4) *DBLP* [17] is a heterogeneous bibliographic graph with typed nodes and relations, where nodes represent authors, papers, venues, and terms, and edges capture their interactions. We consider the task of author node classification. Since the prediction target is the author

Dataset	|V|	|E|	# node types	# attributes
BAShape [43]	2,020,000	12,055,704	4	1
Cora [33]	2,708	5,429	7	1,433
DBLP [17]	26,128	296,563	4	334
ATLAS [3]	59,148	40,823	2	64
MUTAG [10]	17.9(avg.)	19.8(avg.)	7	7
ogbn-Papers100M [22]	111M	1.6B	40	128
Tree-Cycles [43]	67M	1.4B	5	5

Table 1: Summary of datasets

type, we report the author feature dimension (334) in Table 1, while other node types use type-specific features. (5) *BAShape* [43] is a synthetic dataset generated by attaching house-shaped motifs to a Barabási–Albert base graph. A node label corresponds to structural roles (top, middle, bottom, or background). (6) *ogbn-Papers100M* [22] is a large-scale citation network with 1.6 billion citations (edges). Each paper has a 128-dimensional feature vector and is classified into one of 40 research fields. (7) *Tree-Cycles* [43] is a synthetic dataset with hierarchical graphs combining trees and cyclic structures, with 1.4 billion edges, and nodes in five categories represented by 5-dimensional one-hot features. Among these datasets, *DBLP* is used both in the overall efficiency & effectiveness evaluation and as the primary benchmark for detailed sensitivity analysis.

Constraint Generation. For each dataset, we derive constraints from the training portion only. For node-level datasets, constraints are constructed from local L-hop neighborhoods induced from labeled training nodes; for graph-level datasets, constraints are derived directly from the training graphs. We extract typed local patterns from these training-derived structures and instantiate constraints in the form $\varphi = (P, c)$, where P is the antecedent and c is a single-edge consequent. To simulate incomplete observations, we construct "observed" input graphs by masking a fraction of edges as a preprocessing step; the default mask ratio is 5%, and we vary it up to 20% in sensitivity studies. This avoids leakage from test workloads. This setting avoids directly constructing constraints from test workloads and better reflects the practical scenario where constraints are derived from historical or verified knowledge.

Graph Models. We use three representative GNN families in our experiments: GCNs [24], GATs [39], and GraphSAGE [18]. All models are implemented in PyTorch Geometric with a hidden dimension of 32, and the model depth follows the experimental setting. We use the standard train/validation/test split associated with each dataset and configuration, and train each model for 200 epochs using Adam optimizer ($lr = 0.01$). All GNN models achieve competitive predictive performance, with test accuracy ranging from 82.4% to 91.1%. Nevertheless, our method is model-agnostic: given a fixed pretrained model, it aims to produce grounded witnesses that remain faithful to the model's actual behavior, regardless of the underlying architecture or prediction accuracy.

Algorithms. We implemented naiveChase, ApxIChase, and HeuIChase. These three methods share the same overall objective but differ in candidate witness generation and grounding realization. We compare them with two representative post-hoc GNN explainers whose explanation subgraphs can be extracted for a fair comparison. (1) *GNNExplainer* [43] is an established post-hoc method that identifies subgraphs most influential to the model's prediction by optimizing a continuous edge mask through gradient-based learning. (2) *PGExplainer* [30] is a learning-based method that learns a neural edge generator to produce explanations conditioned on node embeddings. Their generated subgraphs are treated as baseline witnesses for comparison. Unlike our methods, these baselines do not perform the subsequent $G_s \rightarrow G_g$ grounding step; their constraint coverage is therefore evaluated only under strict satisfaction of the generated subgraphs. In addition, to test the scalability, we also implemented paraIChase, a parallelized implementation of Inf-Chase in a shared memory, but by partitioning the inference query workload (see details in appendix [2]).

Evaluation Metrics. We report both the efficiency and effectiveness of all methods. (1) For *Efficiency*, for our methods (e.g., ApxIChase, HeuIChase, and naiveChase), we report the total runtime including candidate generation, verification, window maintenance, and grounding-related evaluation, until no further update can be made. This runtime excludes the preprocessing time used to construct the shared input graph. For the baseline GNN explainers, we report the total runtime required to generate explanation subgraphs under the same input graph, model, and task setting. (2) For *Effectiveness*, we evaluate *Coverage*, *Conciseness*, and *Fidelity*. Coverage and conciseness are defined in Section 3.

In particular, we report *normalized coverage* as the primary coverage metric. It focuses on the subset of constraints that are active for the current workload, namely those whose consequent can be matched and whose antecedent is already node-complete in the current input graph, and measures how many of them are successfully grounded within budget k. For fidelity, following conventional post-hoc explanation methods [30, 43], we adopt the standard probability-difference fidelity measure and report its normalized form so that higher values indicate better faithfulness:

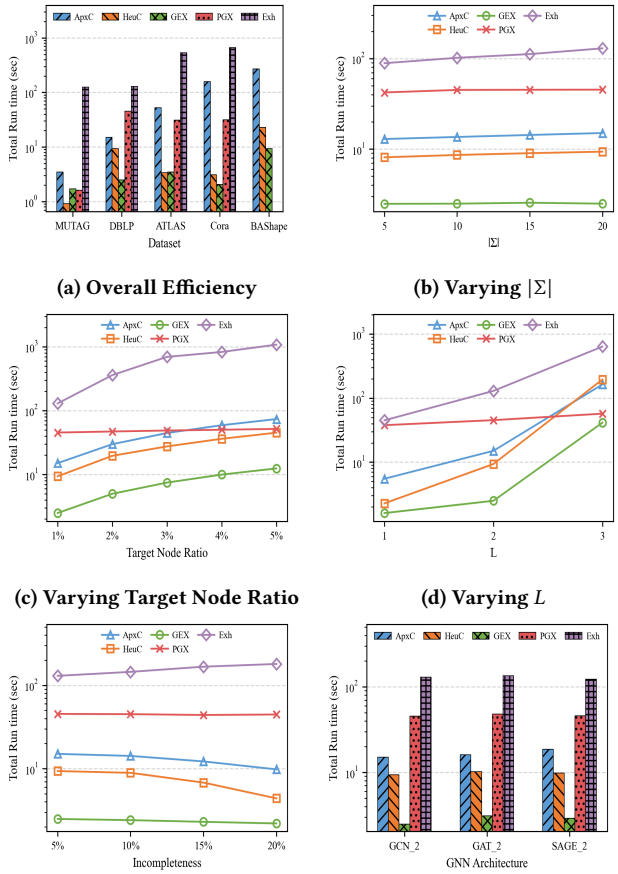
$$\mathsf{Fid}(G_s) = 1 - \left|\Pr\big(Q(M, v_t, G)\big) - \Pr\big(Q(M, v_t, G_s)\big)\right|,$$

where $\Pr(\cdot)$ denotes the predicted probability of the target class. A larger $\mathsf{Fid}(G_s)$ indicates that the witness subgraph preserves the model's prediction more faithfully, *i.e.,* the predicted probability under G_s is closer to that under the original graph G.

Environments. All experiments are conducted on CWRU HPC cluster. Each compute node has an Intel Xeon Silver 4216 CPU at 2.10GHz (16 cores, 32 threads), 125GB of RAM, and a Tesla V100-SXM2 GPU with 32GB of memory. SLURM allocates access to a single node per test. Parallel variants utilize up to 20 CPU threads with GPU acceleration using NVIDIA driver version 560.35, and CUDA version 12.6. The code is available at [1].

7.2 Experimental Results

Exp-1: Efficiency. We compare the runtime of all methods on the benchmark datasets under their default model settings. For simplicity, we abbreviate the methods as follows: ApxC (ApxIChase), HeuC (HeuIChase), GEX (GNNExplainer), PGX (PGExplainer), and Exh (exhaustive naiveChase). Unless explicitly varied, we use the default setting ($K = 6$, $k = 2$, $L = 2$, incompleteness = 5%, $\alpha = \beta = \frac{1}{2}$, $\gamma = 1$). $|\Sigma|$ is dataset dependent [2]. For DBLP, the default setting is $|\Sigma| = 20$. For node-level datasets, the default sampled target ratio is $\frac{|V_t|}{|V|} = 1\%$, where *incompleteness* denotes the proportion of masked edges and $|V_t|$ the number of sampled test nodes.



(a) Overall Efficiency **(b) Varying $|\Sigma|$**

(c) Varying Target Node Ratio **(d) Varying L**

(e) Varying Incompleteness **(f) Varying model type**

Figure 7: Performance Evaluation: Efficiency

Overall Efficiency. Fig. 7(a) reports the total runtime across datasets. Our methods substantially improve over naiveChase (Exh), whose exhaustive constraint enforcement incurs high overhead. For example, on Cora, ApxIChase and HeuIChase are about 4.2× and 214× faster than naiveChase, respectively; on MUTAG, the speedups are about 35.7× and 137×. These gains come from bounded enforcement and more efficient candidate generation. Among our methods, HeuIChase is consistently the fastest due to its constrained search strategy, while ApxIChase incurs moderate overhead from more general exploration. Among the baselines, GNNExplainer (GEX) is the fastest, whereas PGExplainer (PGX) is more expensive due to global training and does not scale to large datasets such as BAShape.

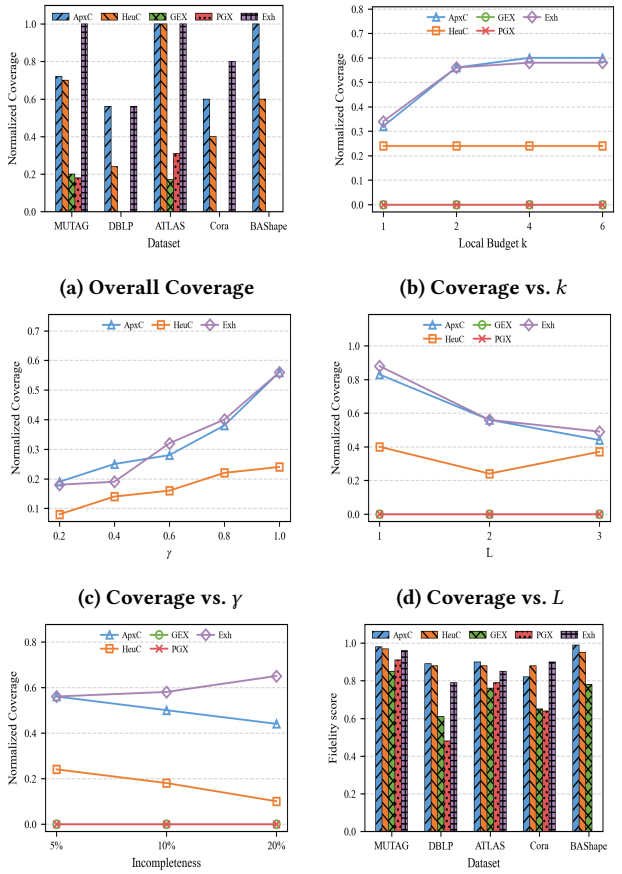
Impact of $|\Sigma|$. We vary the total number of data constraints $|\Sigma| \in \{5, 10, 15, 20\}$. As shown in Fig. 7(b), ApxIChase and HeuIChase exhibit only mild runtime growth as $|\Sigma|$ increases, since their cost is governed mainly by the number of activated constraints rather than the full constraint pool. This trend is gradual across the entire range: the runtime of ApxIChase increases from 12.9s to 15.03s, while that of HeuIChase increases from 8.1s to 9.35s. In contrast, naiveChase is highly sensitive to $|\Sigma|$, as more constraints directly increase the cost of exhaustive fixpoint enforcement. GNNExplainer and PGExplainer remain largely insensitive because they do not perform post-hoc constraint checking or grounding.

Varying the Size of Test Set $|V_t|$. We vary the proportion of target nodes from 1% to 5% on DBLP while fixing other parameters. As shown in Fig. 7(c), all methods become slower as $|V_t|$ grows, since each target node induces an independent workload. This trend is especially clear for ApxIChase, HeuIChase, and GNNExplainer: their runtimes increase from 15.03s to 73.7s, from 9.35s to 45.1s, and from 2.49s to 12.43s, respectively, as the target ratio grows from 1% to 5%. Both ApxIChase and HeuIChase scale approximately linearly, with HeuIChase consistently faster due to cheaper per-target processing. GNNExplainer also grows nearly linearly. By contrast, PGExplainer is less sensitive to $|V_t|$ because its dominant cost lies in shared training, while naiveChase grows the fastest due to repeated exhaustive enforcement for each target. At the same time, PGExplainer increases only modestly, from 45.33s to 51.7s, whereas naiveChase rises sharply from 129.7s to 1080.0s.

Varying GNN *Layer* L. We use GCN and vary the number of layers $L \in \{1, 2, 3\}$ on DBLP. As shown in Fig. 7(d), the runtime of all methods increases with L, since larger L induces larger local neighborhoods. HeuIChase remains relatively efficient, but its runtime still rises from 2.27s at $L = 1$ to 9.35s at $L = 2$, and to 195.8s at $L = 3$, slightly exceeding ApxIChase at the largest depth. This suggests that the Edmonds-based arborescence step can still become a bottleneck on large dense multi-hop neighborhoods, as the constructed arborescence may itself be too large and dense. ApxIChase and naiveChase also slow down with larger L, while PGExplainer remains relatively stable because its dominant cost comes from the shared explainer model rather than per-instance grounding.

Varying Incompleteness. We vary the degree of graph incompleteness from 5% to 20%. As shown in Fig. 7(e), the runtime of ApxIChase, HeuIChase, and GNNExplainer decreases as the input graph becomes sparser, since fewer observed structures remain for witness generation and grounding. This decrease is gradual but consistent: ApxIChase drops from 15.03s to 9.8s, HeuIChase from 9.35s to 4.4s, and GNNExplainer from 2.49s to 2.2s as incompleteness increases from 5% to 20%. In contrast, naiveChase becomes slower as incompleteness increases, because missing structure imposes a heavier exhaustive repair burden. PGExplainer remains largely stable, as its dominant cost comes from the shared explainer model rather than per-instance structural repair. Its runtime stays within a narrow range, from 45.33s to 44.6s, across all four settings.

Varying GNN *Model Type.* We evaluate all methods on DBLP using three representative backbones: 2-layer GCN, GAT, and GraphSAGE. As shown in Fig. 7(f), ApxIChase and HeuIChase remain relatively stable across different backbones, indicating that their cost is dominated more by candidate generation and grounding than by the backbone itself. In particular, ApxIChase shows only a mild increase from GCN to GAT and GraphSAGE, while HeuIChase remains in a similarly narrow range across all three settings. The same trend holds for GNNExplainer, PGExplainer, and naiveChase, all of which show only moderate variation across architectures. Although the absolute runtime changes slightly from one backbone to another, the gap between methods remains broadly consistent. Overall, the relative runtime ordering remains largely unchanged, indicating that runtime differences are driven mainly by algorithmic design rather than by the choice of GNN backbone.



(a) Overall Coverage (b) Coverage vs. k

(c) Coverage vs. γ (d) Coverage vs. L

(e) Coverage vs. Incompleteness (f) Overall Fidelity

Figure 8: Performance Evaluation: Effectiveness

Exp-2: Effectiveness. We next evaluate witness quality in terms of conciseness, coverage, and fidelity. Unless otherwise specified, all results use the default setting in Section 7.1. Overall effectiveness comparisons are reported across datasets, while detailed sensitivity analyses are conducted on *DBLP*.

Overall Coverage. Fig. 8(a) compares the normalized coverage across datasets. Overall, the grounding-aware methods substantially outperform the baseline explainers, confirming the benefit of coupling witness generation with grounding. On *ATLAS*, all three grounding methods achieve full coverage; on *MUTAG*, ApxIChase and HeuIChase remain high (0.72 and 0.70), while naiveChase reaches 1.0. Across datasets, ApxIChase is the most robust overall, e.g., on *DBLP* it clearly outperforms HeuIChase (0.56 vs. 0.24). Even naiveChase does not always achieve the best coverage, suggesting that more aggressive repair does not uniformly yield broader grounding. In contrast, GNNExplainer and PGExplainer generally achieve low or zero coverage, since they do not perform grounding.

Coverage vs. Budget k. We study how normalized coverage changes with the local budget $k \in \{1, 2, 4, 6\}$ on *DBLP*. As shown in Fig. 8(b), increasing k from 1 to 2 clearly improves coverage for ApxIChase and naiveChase, while further increases yield only marginal gains. This suggests that most active constraints that can be grounded by explicit repair already require only a small local budget, and that the remaining uncovered ones are mainly limited by structural mismatch rather than insufficient edge insertions. In contrast, HeuIChase remains flat across all values of k, consistent with its current heuristic design based on arborescence construction rather than an explicit k-sensitive L-Backchase step. The two post-hoc baselines, GNNExplainer and PGExplainer, stay at zero coverage throughout, since they do not perform the grounding step.

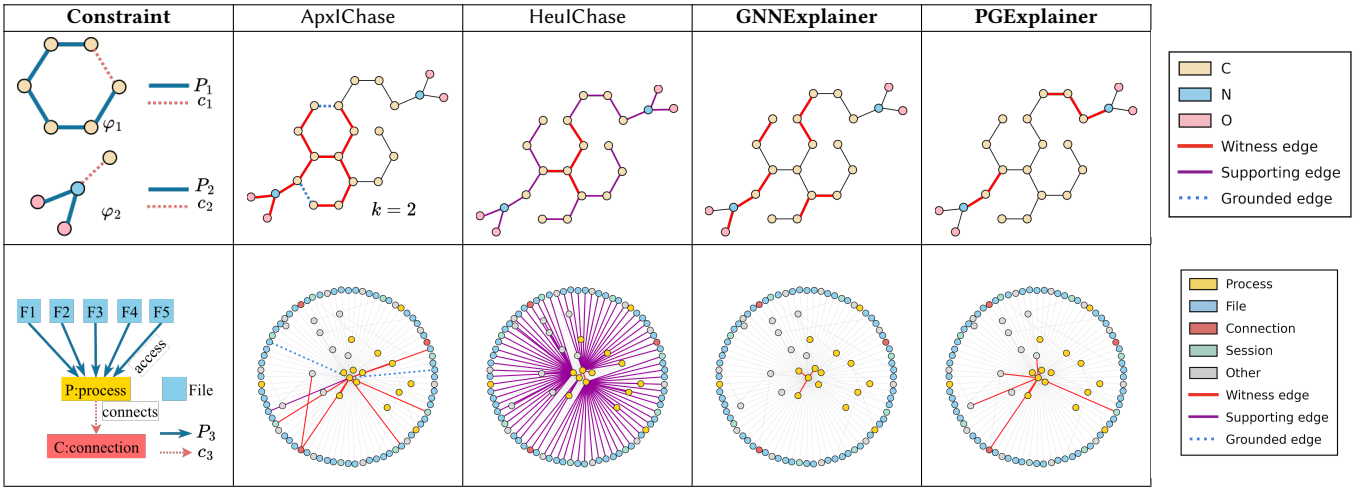
Coverage vs. γ. We study how normalized coverage changes as the weighting parameter $\gamma \in \{0.2, 0.4, 0.6, 0.8, 1\}$ varies on *DBLP*. As shown in Fig. 8(c), increasing γ improves coverage for all three grounding methods. This confirms that placing more emphasis on collective constraint coverage in the set-level objective helps top-K selection. The gain is most pronounced for ApxIChase and naiveChase, while HeuIChase also improves but more mildly, indicating that the objective can help select better candidates even when the candidate space is structurally constrained. Overall, this experiment shows that the set-level coverage term is effective in steering witness selection toward better-grounded results.

Coverage vs. L. We analyze how normalized coverage changes with model depth $L \in \{1, 2, 3\}$ on *DBLP*. As shown in Fig. 8(d), the normalized coverage of ApxIChase and naiveChase decreases as L increases. For example, ApxIChase drops from 0.83 at $L = 1$ to 0.56 at $L = 2$ and 0.44 at $L = 3$, while naiveChase decreases from 0.88 to 0.56 and then 0.49. This is because larger L induces much larger local neighborhoods and activates many more constraints, while the number of constraints that can be covered does not increase at the same rate. For HeuIChase, the trend is less regular, but its overall coverage remains below ApxIChase, reflecting that larger L-hop neighborhoods do not necessarily benefit tree-biased candidate generation. Its coverage changes from 0.40 to 0.24 and then slightly rebounds to 0.36, suggesting that the tree-biased search remains sensitive to how the local neighborhood expands as L grows.

Coverage vs. Incompleteness. We vary graph incompleteness from 5% to 20% missing edges on *DBLP*. As shown in Fig. 8(e), the normalized coverage of ApxIChase and HeuIChase decreases as the input graph becomes sparser, since fewer observed structures remain available for witness generation and grounding. This shows that effective grounding still depends strongly on the quality of the observed local evidence, rather than only on the ability to add missing edges. In contrast, naiveChase improves as incompleteness increases, indicating that exhaustive repair becomes relatively more effective when more structure is missing. GNNExplainer and PGExplainer remain at zero coverage throughout.

Overall Fidelity. Fig. 8(f) reports the overall Fid scores (higher is better) across datasets. ApxIChase and HeuIChase maintain high fidelity, showing that the witnesses selected by our methods still preserve the model's prediction behavior well. Their advantage is most evident on *MUTAG* and *BAShape*, where both methods achieve near-perfect scores, suggesting that, when local patterns are more explicit, grounding-aware selection more easily identifies faithful witnesses. On *DBLP* and *ATLAS*, our methods also consistently outperform GNNExplainer and PGExplainer. Interestingly, naiveChase does not always achieve the best fidelity, even though it performs more aggressive repair. This suggests that stronger

Table 2: Case study: Application of Grounded Witness Generation for Molecular property analysis and Cyberattack detection.



constraint enforcement does not necessarily lead to more faithful witnesses, and may bias witness selection toward structures that are less aligned with the model's actual decision process.

Exp-3: Scalability Test. We evaluate the scalability of our parallel framework (paraIChase) on both large-scale real and synthetic graphs; full algorithmic details are provided in the full version [2]. Each experiment adopts $L = 2$, $k = 2$, $K = 6$, and 100 target nodes by default. A coordinator induces all L-hop subgraphs and assigns them to p processors using load-balanced partitioning by subgraph size. Overall, our methods scale well even on billion-scale graphs. We defer the detailed analysis to appendix/full version [2].

Summary. Overall, ApxIChase provides the strongest overall balance between computational efficiency, grounded coverage, and witness faithfulness, while HeuIChase remains suitable for fast grounded witness generation on large and tree-like graphs.

Exp-4: Case Studies. We next showcase how our methods effectively link predictions to real data evidence declared by constraints.

Case I: Grounding Molecular Property Analysis. Table 2 (Row 1) shows a representative case from *MUTAG*. The leftmost column gives two constraints. Constraint φ_1 specifies a six-node ring pattern, where the antecedent P_1 is the solid blue path and the consequent c_1 is the missing ring-closing edge shown in red dotted style. Constraint φ_2 specifies a three-edge local functional pattern, where P_2 consists of the two solid blue edges and c_2 is the red dotted edge incident to the upper node. Starting from an incomplete molecular graph, ApxIChase produces a connected witness (red) that captures the branch pattern of φ_2, and uses one grounded edge (blue dotted) within budget k=2 to complete an additional ring structure. HeuIChase instead yields a more tree-structured witness and expands it with supporting edges (purple) into a larger grounded graph that grounds φ_2. In contrast, GNNExplainer and PGExplainer recover only partial substructures from the left ring and the side branch, leaving the ring-closure constraints unsatisfied. As a result, the returned subgraphs fail to capture the complete constrained pattern. This example shows that our methods lift model-faithful witnesses into constraint-aligned grounded graphs, while the baselines remain fragmented and ungrounded.

Case II: Data Exfiltration Attack Investigation. We analyze a backdoor process flagged by a 3-layer GraphSAGE model for credential theft and data exfiltration. Table 2 (Row 2) compares reconstructed attacks under constraint $\varphi_3 : P_3 \rightarrow c_3$, which requires that any process accessing five sensitive files—F1:/etc/passwd, F2:.ssh/id_rsa, F3:cookies.db, F4:confidential.pdf, and F5:credentials.txt—must also show the outbound connection C:198.51.100.25:443. ApxIChase yields a witness centered on the suspicious process, where G_s already contains the outbound connection and three file-access edges; backchase then introduces two grounded edges (blue dotted) to complete P_3, so the resulting G_g grounds φ_3 and captures both credential collection and exfiltration. HeuIChase also identifies the same central witness region, but instead of explicitly adding grounded edges, it expands the witness with supporting edges (purple) into a dense arborescence that still grounds φ_3. In contrast, the witness generated by GNNExplainer covers neither the consequent c_3 nor any edge in the antecedent pattern P_3; it contains only process-to-process access edges and is therefore semantically misaligned with φ_3. The witness generated by PGExplainer covers the consequent c_3 (the process-to-outbound-connection edge), but does not cover any edge in P_3. This example shows that our methods can connect model-faithful witnesses to the attack rule, whereas the post-hoc explainer baselines remain incomplete or misaligned under the constraint.

8 CONCLUSION

We have introduced k-grounded witnesses to characterize subgraphs that are necessary to reconstruct graph inference query results, and simultaneously be grounded by graph data constraints. We introduced quality measures for k-grounded witnesses, and verified the hardness of computing the optimal k-grounded witnesses. We present both a principled algorithm inspired by the Chase&Backchase framework, and two practical specifications to efficiently generate k-grounded witnesses as general subgraphs and trees. We have experimentally verified that our algorithms outperform existing graph explanation techniques in generating grounded witnesses and their applications in drug design and cybersecurity.

REFERENCES

[1] 2026. Anonymous Code Repository for Grounding Graph Inference with Data Constraints. https://anonymous.4open.science/r/Grounded_Witness-46EE

[2] 2026. Full version. https://anonymous.4open.science/r/Grounded_Witness-46EE/Grounding_Graph_Inference_Analysis_with_Data_Constraints.pdf

[3] Abdulellah Alsaheel, Yuhong Nan, Shiqing Ma, Le Yu, Gregory Walkup, Z Berkay Celik, Xiangyu Zhang, and Dongyan Xu. 2021. ATLAS: A sequence-based learning approach for attack investigation. In *30th USENIX Security Symposium (USENIX Security 21)*. 3005–3022.

[4] Burouj Armgaan, Manthan Dalmia, Sourav Medya, and Sayan Ranu. 2024. GraphTrail: Translating GNN predictions into human-interpretable logical rules. *Advances in Neural Information Processing Systems* 37 (2024), 123443–123470.

[5] Peter Buneman, Sanjeev Khanna, and Tan Wang-Chiew. 2001. Why and where: A characterization of data provenance. Springer, 316–330.

[6] Marco Calautti, Mostafa Milani, and Andreas Pieris. 2023. Semi-Oblivious Chase Termination for Linear Existential Rules: An Experimental Study. *Proc. VLDB Endow.* 16, 11 (2023), 2858–2870.

[7] Ming Chen, Zhewei Wei, Bolin Ding, Yaliang Li, Ye Yuan, Xiaoyong Du, and Ji-Rong Wen. 2020. Scalable graph neural networks via bidirectional propagation. *Advances in neural information processing systems* 33 (2020), 14556–14566.

[8] Tingyang Chen, Dazhuo Qiu, Yinghui Wu, Arijit Khan, Xiangyu Ke, and Yunjun Gao. 2024. View-based explanations for graph neural networks. *Proceedings of the ACM on Management of Data* 2, 1 (2024), 1–27.

[9] James Cheney, Laura Chiticariu, Wang-Chiew Tan, et al. 2009. Provenance in databases: Why, how, and where. *Foundations and Trends in Databases* (2009).

[10] Asim Kumar Debnath, Rosa L Lopez de Compadre, Gargi Debnath, Alan J Shusterman, and Corwin Hansch. 1991. Structure-activity relationship of mutagenic aromatic and heteroaromatic nitro compounds. correlation with molecular orbital energies and hydrophobicity. *Journal of medicinal chemistry* 34, 2 (1991), 786–797.

[11] Alin Deutsch, Lucian Popa, and Val Tannen. 2001. Chase & backchase: A method for query optimization with materialized views and integrity constraints. (2001).

[12] Jinlong Du, Senzhang Wang, Hao Miao, and Jiaqiang Zhang. 2021. Multi-Channel Pooling Graph Neural Networks.. In *IJCAI*. 1442–1448.

[13] Jack Edmonds et al. 1967. Optimum branchings. *Journal of Research of the national Bureau of Standards B* 71, 4 (1967), 233–240.

[14] Wenfei Fan and Ping Lu. 2019. Dependencies for graphs. *ACM Transactions on Database Systems (TODS)* 44, 2 (2019), 1–40.

[15] Wenfei Fan, Xin Wang, Yinghui Wu, and Jingbo Xu. 2015. Association rules with graph patterns. *Proceedings of the VLDB Endowment (PVLDB)* 8, 12 (2015), 1502–1513.

[16] Wenfei Fan, Yinghui Wu, and Jingbo Xu. 2016. Functional dependencies for graphs. In *Proceedings of the International Conference on Management of Data (SIGMOD)*. 1843–1857.

[17] Xinyu Fu, Jiani Zhang, Ziqiao Meng, and Irwin King. 2020. Magnn: Metapath aggregated graph neural network for heterogeneous graph embedding. In *Proceedings of the Web Conference 2020*. 2331–2341.

[18] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. *NeurIPS* (2017).

[19] Myoungji Han, Hyunjoon Kim, Geonmo Gu, Kunsoo Park, and Wook-Shin Han. 2019. Efficient subgraph matching: Harmonizing dynamic programming, adaptive matching order, and failing set together. In *Proceedings of the International Conference on Management of Data (SIGMOD)*. 1429–1446.

[20] Wook-Shin Han, Jinsoo Lee, and Jeong-Hoon Lee. 2013. Turboiso: towards ultrafast and robust subgraph isomorphism search in large graph databases. In *Proceedings of the International Conference on Management of Data (SIGMOD)*. 337–348.

[21] Yuxing Han, Ziniu Wu, Peizhi Wu, Rong Zhu, Jingyi Yang, Liang Wei Tan, Kai Zeng, Gao Cong, Yanzhao Qin, Andreas Pfadler, Zhengping Qian, Jingren Zhou, Jiangeng Li, and Bin Cui. 2022. Cardinality Estimation in DBMS: A Comprehensive Benchmark Evaluation. *Proceedings of the VLDB Endowment* 15, 4 (2022), 752–765. https://doi.org/10.14778/3503585.3503586

[22] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open graph benchmark: Datasets for machine learning on graphs. *NeurIPS* (2020).

[23] Richard M Karp. 2009. Reducibility among combinatorial problems. In *50 Years of Integer Programming 1958-2008: from the Early Years to the State-of-the-Art*. Springer, 219–241.

[24] Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).

[25] Douglas J Klein. 2010. Centrality measure in graphs. *Journal of mathematical chemistry* 47, 4 (2010), 1209–1223.

[26] Ryoji Kubo and Djellel Difallah. 2025. RAW-Explainer: Post-hoc Explanations of Graph Neural Networks on Knowledge Graphs. *arXiv preprint arXiv:2506.12558* (2025).

[27] Wanyu Lin, Hao Lan, and Baochun Li. 2021. Generative causal explanations for graph neural networks. In *International conference on machine learning*. PMLR, 6666–6679.

[28] Wanyu Lin, Hao Lan, Hao Wang, and Baochun Li. 2022. Orphicx: A causality-inspired latent variable model for interpreting graph neural networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 13729–13738.

[29] Ana Lucic, Maartje A Ter Hoeve, Gabriele Tolomei, Maarten De Rijke, and Fabrizio Silvestri. 2022. Cf-gnnexplainer: Counterfactual explanations for graph neural networks. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 4499–4511.

[30] Dongsheng Luo, Wei Cheng, Dongkuan Xu, Wenchao Yu, Bo Zong, Haifeng Chen, and Xiang Zhang. 2020. Parameterized explainer for graph neural network. *Advances in neural information processing systems* 33 (2020), 19620–19631.

[31] Michael J Maher and Divesh Srivastava. 1996. Chasing constrained tuple-generating dependencies. In *Proceedings of the fifteenth ACM SIGACT-SIGMOD-SIGART symposium on Principles of database systems*. 128–138.

[32] Daniel Mawhirter, Sam Reinehr, Connor Holmes, Tongping Liu, and Bo Wu. 2021. Graphzero: A high-performance subgraph matching system. *ACM SIGOPS Operating Systems Review* 55, 1 (2021), 21–37.

[33] Andrew Kachites McCallum, Kamal Nigam, Jason Rennie, and Kristie Seymore. 2000. Automating the construction of internet portals with machine learning. *Information Retrieval* (2000).

[34] George L Nemhauser, Laurence A Wolsey, and Marshall L Fisher. 1978. An analysis of approximations for maximizing submodular set functions—I. *Mathematical programming* 14, 1 (1978), 265–294.

[35] Paolo Pareti, George Konstantinidis, Timothy J Norman, and Murat Şensoy. 2019. SHACL constraints with inference rules. In *International Semantic Web Conference*. Springer, 539–557.

[36] Yufan Sheng, Xin Cao, Kaiqi Zhao, Yixiang Fang, Jianzhong Qi, Wenjie Zhang, and Christian S. Jensen. 2025. ACE: A Cardinality Estimator for Set-Valued Queries. *Proceedings of the VLDB Endowment* 18, 7 (2025), 2112–2125. https://doi.org/10.14778/3734839.3734848

[37] Larissa C Shimomura, George Fletcher, and Nikolay Yakovets. 2020. GGDs: graph generating dependencies. In *CIKM*.

[38] Qi Song, Mohammad Hossein Namaki, Peng Lin, and Yinghui Wu. 2020. Answering why-questions for subgraph queries. *IEEE Transactions on Knowledge and Data Engineering* 34, 10 (2020), 4636–4649.

[39] Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, Yoshua Bengio, et al. 2017. Graph attention networks. *stat* (2017).

[40] Varvara Voinarovska, Mikhail Kabeshov, Dmytro Dudenko, Samuel Genheden, and Igor V Tetko. 2023. When yield prediction does not yield prediction: an overview of the current challenges. *Journal of Chemical Information and Modeling* 64, 1 (2023), 42–56.

[41] Minh Vu and My T Thai. 2020. Pgm-explainer: Probabilistic graphical model explanations for graph neural networks. *Advances in neural information processing systems* 33 (2020), 12225–12235.

[42] Xiang Wang, Yingxin Wu, An Zhang, Fuli Feng, Xiangnan He, and Tat-Seng Chua. 2022. Reinforced causal explainer for graph neural networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2022), 2297–2309.

[43] Zhitao Ying, Dylan Bourgeois, Jiaxuan You, Marinka Zitnik, and Jure Leskovec. 2019. Gnnexplainer: Generating explanations for graph neural networks. *Advances in neural information processing systems* 32 (2019).

[44] Hao Yuan, Haiyang Yu, Jie Wang, Kang Li, and Shuiwang Ji. 2021. On explainability of graph neural networks via subgraph explorations. In *International conference on machine learning*. PMLR, 12241–12252.

[45] Hongkuan Zhou, Ajitesh Srivastava, Hanqing Zeng, Rajgopal Kannan, and Viktor Prasanna. 2021. Accelerating large scale real-time GNN inference using channel pruning. *arXiv preprint arXiv:2105.04528* (2021).

[46] Mingzhe Zhu, Liwang Zhu, Huan Li, Wei Li, and Zhongzhi Zhang. 2024. Resistance distances in directed graphs: Definitions, properties, and applications. *Theoretical Computer Science* 1009 (2024), 114700.

[47] Chuanyu Zong, Zefang Dong, Xiaochun Yang, Bin Wang, Tao Qiu, and Huaijie Zhu. 2023. Efficiently answering why-not questions on radius-bounded k-core searches. In *International Conference on Database Systems for Advanced Applications*. Springer, 93–109.

[48] Chuanyu Zong and Chengwei Zhang. 2024. Effectively answering why questions on structural graph clustering. *Applied Soft Computing* 154 (2024), 111405.

9 APPENDIX

9.1 Appendix A: Formal Notations and Proofs

Graph Data Constraints: Specifications. The constraint set Σ contains a set of graph data constraints, where a data constraint $\varphi = (P, c)$ can be easily specified to the following types of constraints.

(1) A graph functional dependency (GFD) [16]. A GFD φ has a general form $(Q, X \rightarrow Y)$, where Q is a graph pattern query with a set of pattern nodes and edges, and each pattern node u is associated with a variable x_u from a finite set of variables U, and $X \rightarrow Y$ enforces a value binding constraint (such as a functional dependency) over the attribute set $\mathbf{A}$ of the nodes that match node variables in Q via subgraph isomorphism. A GFD φ' can be equivalently expressed by a data constraint $\varphi = (P, c)$, where P contains Q and enforces an equivalent literal $x_u.X = x'_u.X$, over the two nodes in Q with variables involved in X, and c is a triple $(u, \emptyset, u')$ over the same pair of variables u and u', and enforces an equivalent literal $x_u.Y = x'_u.Y$. It expresses the semantics consistently with φ' as: "for any two nodes that match the nodes in pattern P, if they have the same values for X, they must have the same value on attributes Y".

(2) A graph association rule (GPAR) [15] is in the form of $P \rightarrow P'$, where P and P' are graph pattern queries, and the rule enforces the co-occurrence of the matches of P and P' in a graph G. A graph association rule γ can be expressed by a set of graph data constraints Σ, where each data constraint $\varphi = (P, c)$ has a same antecedent P with a shared set of nodes variables, and c is a distinct, single triple pattern (a pattern edge) of P' defined a set of shared variables. A graph $G \models \Sigma$ ensures the co-occurrence of any subgraph G_s that matches P and a corresponding subgraph induced by the edge matches of c conjunctively, over shared node variables.

(3) A graph generation dependency (GGDs) [37] has a form $(Q_s, \phi) \rightarrow (Q_t, \phi')$, defined on the node variables U, where ϕ and ϕ' are a set of differential constraints that specify equality or inequality value constraints over the attributes of nodes that match Q_s and Q_t, respectively. GGDs can be expressed by a set of data constraints Σ in a similar construction in (2), where each constraint $\varphi = (Q_s \cup Q_t), c$ requires the existence of a (possibly disconnected) subgraph that matches Q_s and Q_t simultaneously, and have the values of node matches satisfying ϕ and ϕ' as differential constraints.

(4) Similar, SHACL [35] is subsumed by a practical implementation of data constraints Σ, where a SHACL encodes a graph pattern P that resembles a SPARQL query, and c specifies a conjunction of value-binding constraints for RDF data validation.

In general, our constraint syntax can express TED-/TGD-style graph dependencies over graph data. When c is $\emptyset$, a constraint φ reduces to a graph-pattern query, serving as a basis for representative graph query languages such as SPARQL, Cypher, or Gremlin. Otherwise, c acts as a single consequent pattern (or value-binding condition) associated with P, and the constraint is used as a grounding rule that checks or enforces the co-occurrence of P and c in an auxiliary graph.

Hardness of k-bounded Witness Generation. We provide the following analysis to show the hardness of the witness generation problem. We first start with a hardness analysis for a witness verification problem. Given an inference query $Q = (M, v_t, G)$, a graph ML model M, and a subgraph G_s, it is to verify if G_s is a witness of the result of Q.

Lemma 4: *The witness verification problem is in* PTIME. □

Proof. This result can be verified by constructing a PTIME procedure that invokes the inference process of the fixed graph ML model M, and checks by definition if G_s is either factual or counterfactual, in polynomial time. □

We next consider the k-grounded witness verification problem. Given an inference query $Q = (M, v_t, G)$, a graph ML model M, an auxiliary grounded graph G_g containing a witness G_s of G and a set of grounded edges ΔE, a set of data constraints Σ, and an integer k, it is to verify if G_s is a minimal k-grounded witness of the result of Q.

Lemma 5: *The k-grounded witness verification problem is* coNP-*hard.* □

Proof. The hardness carries over from the hardness of the validation problem of graph functional dependencies (GFD), which is a special case of data constraints. Let G_s be a subgraph that passed the PTIME witness verification problem (see Lemma 4), by calling a GNN inference function. It then suffices to decide if $G_s \models_k \varphi$. Let $k = 0$, and φ is a single graph functional dependency. Then we consider a reduction from a GFD validation problem. The latter is to verify if $G_s \models \varphi$ as a GFD. As the validation problem for GFDs has been verified to be coNP-hard, hence the k-grounded witness verification remains coNP-hard. □

Hardness of Witness generation. We next prove Theorem 1.

Proof. We reduce from the classical *Clique* problem [23]. Given an undirected graph $H = (V_H, E_H)$ and an integer $r \geq 3$, the question is whether H contains a clique of size r.

From this clique instance (H, r), we construct an instance of the decision version of witness generation as follows. The graph G contains a copy of H, where every copied node has the same type C. We then add two fresh nodes: a target node v_t of type T, and an anchor node a of type A. We add the edge (v_t, a), and for every node $u \in V_H$, we add the edge (a, u). If the graph model is directed, each undirected edge of H is replaced by two opposite directed edges; this does not affect the reduction.

We fix a deterministic GNN model M_0 that returns the same output for every input graph containing v_t. Hence the singleton graph G_s consisting only of v_t is a factual witness for $Q(M_0, v_t, G)$; moreover, it is minimal, since it has no edge that can be removed. Therefore the witness requirement is trivial, and the hardness comes entirely from grounding.

We use a single constraint $\Sigma = \{\varphi\}$, where $\varphi = (P, c)$. Let y be a variable of type A, and let $x_1, \ldots, x_r$ be variables of type C. The antecedent P contains (i) the anchor edge (y, x_1), and (ii) all clique edges (x_i, x_j) for $1 \leq i < j \leq r$, except for the pair (x_1, x_r). The consequent is exactly the missing edge

$$c = (x_1, x_r).$$

In the constructed instance, a grounded graph G_g satisfies φ if and only if it contains a match of an r-clique in the copy of H, together with the anchor edge from a to one clique node.

We now prove correctness of the reduction.

($\Rightarrow$) Suppose that H contains a clique $\{u_1, \ldots, u_r\}$ of size r. Let G_g be the connected subgraph of G induced by $\{v_t, a, u_1, \ldots, u_r\}$. Under the homomorphism $h(y) = a$ and $h(x_i) = u_i$ for $1 \leq i \leq r$, every edge of P and the consequent edge c is present in G_g. Hence

$G_g \models \varphi$. Since $k = 0$, no grounded edge needs to be added. Therefore (G_s, G_g) is a feasible grounded witness pair with $K = 1$.

($\Leftarrow$) Conversely, suppose that there exists a feasible grounded witness pair (G'_s, G_g) with $K = 1$, $k = 0$, and $G_g \models \varphi$. Since y has type A, and a is the only node of type A in G, the match of y must be a. Likewise, since $x_1, \ldots, x_r$ all have type C, they must be mapped to nodes in the copy of H. Because all edges in P and the consequent edge c are present in G_g, the images of $x_1, \ldots, x_r$ are pairwise adjacent in the copy of H, and thus form a clique of size r. Moreover, these images must be distinct: if two variables were mapped to the same node, then some required edge in P or the consequent c would become a self-loop, but H has no self-loops.

Hence, the constructed instance admits a feasible grounded witness pair if and only if H contains a clique of size r. The construction is polynomial in the size of (H, r). Therefore the decision version of the witness generation problem is NP-hard, even when M is fixed, $K = 1$, $k = 0$, and $|\Sigma| = 1$. □

Properties of I-Chase. We next provide proof to show the properties of I-Chase. We provide the detailed proof of Lemma 2. Before we present the formal proof, we introduce several notations below.

In an analogy of Chase that enforces data constraints to relational tables, we consider I-Chase an extended Chase process over graph G and Σ to "transform" G to a chase-saturated graph G', *i.e.,* no further applicable constraint can be enforced on G'. We formalize the computation of I-Chase below. A I-Chase is a rewriting process $< \mathcal{S}, \mathcal{E}, \mathcal{O} >$ that consists of the following.

(1) $\mathcal{S}$ is a set of states, and each state s is a pair (Σ', G_s) such that $\Sigma' \subseteq \Sigma$, and G_s is a subgraph of G.

(2) $\mathcal{O} \subseteq O_\Sigma \times O_G$ is a set of operators, where each operator o is a pair (o_Σ, o_G). Here an operator o_Σ is from a set $\{\oplus\varphi, \ominus\varphi$, for a constraint $\varphi \in \Sigma$; and $\oplus\varphi$ (resp. $\ominus\varphi$) is to "add" (resp. "remove") a constraint to (resp. from) a current subset $\Sigma' \subseteq \Sigma$. o_G is from a set $\{\oplus e, \ominus e\}$ for an edge $e \in E$ of G; which is either an edge insertion ($\oplus e$) or deletion ($\ominus e$) to be applied to a current subgraph $G' \subseteq G$. Both o_Σ and o_G can be $\emptyset$.

(3) $\epsilon_o \in \mathcal{E}$ is a transition between two states $s_1 = (\Sigma_1, G_{s_1})$ and $s_2 = (\Sigma_2, G_{s_2})$. A transition $\epsilon_o(s_1, s_2)$ applies an operator $o = (o_\Sigma, o_G)$ simultaneously applies o_Σ to Σ_1 to update it to Σ_2, and o_G to G_{s_1} to G_{s_2}, thus transits from state s_1 to s_2.

A state $s = (\Sigma_s, G_s)$ is a *terminating* state, if (1) G_s is a witness *w.r.t.* the input inference query Q, and (2) $G_s \models \Sigma_s$.

I-Chase *Sequence.* A *sequence* ρ of I-Chase is a sequential, consecutive processing of transitions ($\rho = \{\epsilon_1, \ldots \epsilon_n\}$) from a starting state s_1 to a *result* state s_n. A sequence ρ is *terminating*, if its result s_n is a terminating state.

Runs of I-Chase. A running of I-Chase starts from an initial state $s_0 = (\Sigma, G)$. The running spawns multiple transitions at a state s with applicable operators. The running is terminating, if it contains at least one terminating sequence.

We start with an intuitive explanation below.

Termination. The vertex set $N^L(v_t)$ is finite, bounding possible edges by $|N^L(v_t)|$. I-Chase adds edges monotonically (*i.e.,* $G^{(i)} \subseteq G^{(i+1)}$) and never delete edges. Since only finitely many edges can be added, the fixed-point computation must reach a saturated graph G' where no constraint $\varphi \in \Sigma$ is applicable. Hence, I-Chase terminates after a finite number of enforcement steps.

Church-Rosser. A rewriting system is Church-Rosser if all execution sequences converge to a common extension. We prove this via: (1) Local Confluence: For any graph G^i during execution, suppose $\varphi_1 = (P_1, c_1)$ and $\varphi_2 = (P_2, c_2)$ are applicable. Adding edge e_{c_1} does not affect the match of P_2(Σ contains only positive conditions, no deletions.) Applying both yields $G^{(i)} \oplus \{e_{c_1}, e_{c_2}\}$ regardless of order. Hence all critical pairs merge. (2) Global Confluence: I-Chase is terminating and locally confluent. By Newman's Lemma, it is globally confluent: and two maximally executions from $G^{(0)}$ produce isomorphic saturated graphs satisfying Σ. Confluence directly implies the Church-Rosser property.

We provide a more formal proof below. The general proof idea is to represent I-Chase over Σ and G into an equivalent Chase process, which the latter can equivalently simulate as a reasoning system over a finite relational database. (1) We represent G as a table with a universal schema that contains a source node ID, a target node ID, and all nodes and edge attributes. (2) We rewrite each constraint φ as either a TEG for any literal that enforces an equality condition in c, and a TGD, for any c with non-empty u_p, to enforce a new tuple in accordance with the insertion of a new edge in G. (3) The additional literals from P are treated as data constraints posed by the head of the TGDs or TEDs, defined on the node variables. We can then verify that the Church-Rosser property holds for the above Chase process, by reducing it to an equivalent, special case of the Chase process for graph dependencies [14], which has been shown to have Church-Rosser property.

Lemma 6: *The Church-Rosser Property does not hold for* L-Chase. □

Proof. Unlike I-Chase, which guarantees convergence to a unique fixed point regardless of the enforcement order, L-Chase imposes a strict limit of L reasoning rounds. This bound forces the procedure to terminate once L rounds are exhausted, potentially before saturation is achieved. Consequently, if multiple constraints are applicable, prioritizing different constraints will produce distinct graph states upon reaching round L. Since the procedure halts at this limit, it precludes the subsequent steps necessary to unify these states and achieve confluence. Thus, the output of L-Chase is order-dependent. □

Axiomatic Analysis. We prove the three properties of the set objective $F(\mathcal{P})$ stated in Section 3. Let $\mathcal{C}$ be the finite set of feasible grounded witness pairs for a fixed inference query $Q(M, v_t, G)$, budget k, and constraint set Σ. Since $|\mathcal{P}| \leq K$ and $\mathcal{C}$ is finite, there exists an optimal witness set $\mathcal{P}^* \in \arg\max_{\mathcal{P} \subseteq \mathcal{C}, |\mathcal{P}| \leq K} F(\mathcal{P})$.

(1) **Scale invariance**. Let $c > 0$ be a constant, and define $F_c(\mathcal{P}) = c \cdot F(\mathcal{P}) = c \cdot \sum_{(G_s, G_g) \in \mathcal{P}} q(G_s, G_g) + c \cdot \gamma \cdot \mathrm{cov}(\mathcal{P})$. For any two feasible witness sets $\mathcal{P}_1, \mathcal{P}_2 \subseteq \mathcal{C}$, $F_c(\mathcal{P}_1) \geq F_c(\mathcal{P}_2) \iff c \cdot F(\mathcal{P}_1) \geq c \cdot F(\mathcal{P}_2) \iff F(\mathcal{P}_1) \geq F(\mathcal{P}_2)$. Hence the maximizer of the objective remains unchanged under a common positive scaling of the component scores and the coverage term. Therefore, $\mathcal{P}^*$ is invariant to such normalization of user preference weights.

(2) **Non-monotonicity with respect to witness / grounding size**. This property concerns the sizes of individual witnesses or

grounded graphs, rather than set inclusion over witness sets. Consider two singleton witness sets $\mathcal{P}_1 = \{(G_s, G_g)\}, \mathcal{P}_2 = \{(G'_s, G'_g)\}$. It is possible that $|G_s| \geq |G'_s|$ and yet $F(\mathcal{P}_1) < F(\mathcal{P}_2)$: for example, the larger witness G_s may have strictly lower conciseness, while yielding no compensating gain in alignment or coverage. Conversely, it is also possible that $|G_s| \geq |G'_s|$ and $F(\mathcal{P}_1) > F(\mathcal{P}_2)$: for example, the larger grounded graph G_g may ground additional constraints or require proportionally fewer inserted edges, thereby increasing the alignment and/or coverage contribution. Hence there is no monotone relationship between the sizes of individual witnesses / grounded graphs and the objective value $F(\mathcal{P})$.

(3) **Locality**. For any selected witness set $\mathcal{P}$, the contribution of each pair $(G_s, G_g) \in \mathcal{P}$ to the objective is captured locally by $q(G_s, G_g)$, which depends only on that witness and its corresponding grounded graph. The only interaction among different selected pairs arises through the set-level coverage term $\text{cov}(\mathcal{P}) = \frac{|\text{CovSet}(\mathcal{P})|}{|\Sigma|}$, $\text{CovSet}(\mathcal{P}) = \bigcup_{(G_s,G_g)\in\mathcal{P}} \Sigma'(G_g)$. Therefore, no information outside the selected grounded witness pairs and their grounded graphs affects the value of $F(\mathcal{P})$. This establishes the locality property.

Remark. The non-monotonicity above concerns changes in the sizes of individual witnesses or grounded graphs. It does not contradict the monotonicity of $F(\mathcal{P})$ as a set function with respect to set inclusion, which is the property used in the approximability analysis.

9.2 Appendix B: Algorithms and Analysis

Implementation of naiveChase. The algorithm naiveChase enforce each graph constraint $\varphi \in \Sigma$ in $G^L(v_t)$.

(1) The I-Chase procedure extends Chase algorithm to perform a fixpoint enforcement of Σ as follows. Whenever there is an applicable constraint $\varphi = (P, c)$, it iteratively retrieves a subgraph $G_\varphi = G_s \oplus e_c$, where G_s is a subgraph of G that matches P (obtained by invoking fast subgraph match enumeration algorithms *e.g.*, [19, 20, 32]), and e_c is a single edge that matches the consequent (triple pattern) c, either inserted (enforced), or originally exists in G.

(2) For each such subgraph $G_\varphi = G_s \oplus e_c$, it invokes a verification process VerifyWitness, which performs the inference of M over G_φ to test whether G_φ is a witness for the inference query $Q(M, v_t, G)$. If so, it adds G_φ as a 0-grounded witness by φ in $G^L(v_t)$.

For each $\varphi \in \Sigma$, it enumerates all candidate 0-grounded witness pairs. It then returns a top-K set that maximizes the overall objective $F(\mathcal{P})$.

Procedure L-Backchase **(line 10, Algorithm 1, Fig. 4)**. Given an inverse constraint set Σ_b, a verified witness G_s, the merged pattern graph G_Σ, the memoization map $\mathcal{M}$, and a budget k, procedure L-Backchase constructs the grounded graph set $\mathcal{G}_g$ used in the Backchase phase. For each applicable inverse constraint $\varphi^{-1} = (c, P) \in \Sigma_b$, it first checks whether the lower-bound budget $\mathcal{M}[i].b$ exceeds k. If not, it derives the missing grounded edges ΔE from the unmatched entries of $\mathcal{M}[i].P$, constructs a candidate grounded graph G_g, and verifies whether the resulting graph contains an occurrence of P_i. Each successful graph is added to $\mathcal{G}_g$, which is then returned to Algorithm 1 to form candidate grounded witness pairs.

Algorithm 3 Procedure L-Backchase $(\Sigma_b, G_s, G_\Sigma, \mathcal{M}, k)$

```
set G_g := ∅;
for each φ^{-1} ∈ Σ_b do
    if M[i].b > k then
        continue;
    derive ΔE from M[i].P ("0" entries);
    construct and verify G_g (w. Weisfeiler-Leman test);
    if G_g contains an occurrence of P_i then
        G_g := G_g ∪ {G_g};
return G_g;
```

Figure 9: Procedure L-Backchase

Algorithm 4 : **Procedure** UpdateWK (C, K)

```
W_K ← ∅
for i = 1 to K do
    for each (G_s, G_g) ∈ C \ W_K, compute Δ((G_s, G_g) | W_K)
    select (G*_s, G*_g) ∈ C \ W_K with the largest Δ((G_s, G_g) | W_K)
    if no such pair exists then
        break
    W_K ← W_K ∪ {(G*_s, G*_g)}
return W_K
```

Figure 10: Procedure UpdateWK

Procedure UpdateWK **(line 12, Algorithm 1, Fig. 4)**. Given the candidate pair pool C of grounded witness pairs generated so far, UpdateWK greedily refreshes the current top-K set under the set objective $F(\mathcal{P})$. At each step, for every candidate pair $(G_s, G_g) \in C \setminus W_K$, it computes the marginal gain

$$\Delta((G_s, G_g) \mid W_K) = q(G_s, G_g) + \gamma \cdot \frac{|\Sigma'(G_g) \setminus \text{CovSet}(W_K)|}{|\Sigma|},$$

and selects the pair with the largest marginal gain to update W_K. Since each iteration inserts at most one previously unselected pair and the procedure performs at most K iterations, the returned window always satisfies $|W_K| \leq K$. Hence, UpdateWK returns, upon request time, a greedy approximation to the best size-K set under $F(\mathcal{P})$ over the grounded witness pairs generated so far.

Optimization: Pruning Σ. To reduce redundant constraint enforcement, ApxIChase applies a structural pruning rule over Σ to avoid repeatedly checking constraints whose antecedents are subsumed by others.

Lemma 7: *Given two constraints $\varphi_i = (P_i, c)$ and $\varphi_j = (P_j, c)$ with the same consequent c, if $P_i \subseteq P_j$, then $G_s \models_k \varphi_j \implies G_s \models_k \varphi_i$.* □

That is, whenever G_s is k-grounded by the stronger constraint φ_j, it is also k-grounded by the weaker constraint φ_i. Hence φ_i is redundant and can be safely pruned.

Proof sketch: Suppose $G_s \models_k \varphi_j$. Then, by definition, there exists a grounded graph $G_g = G_s \oplus \Delta E$ with $|\Delta E| \leq k$ such that $G_g \models \varphi_j$. Hence there exists a homomorphism h such that $h(P_j) \subseteq G_g$ and

$h(c) \in E(G_g)$. Since $P_i \subseteq P_j$, the restriction of h to P_i also yields a match of P_i in G_g, while the same edge $h(c)$ still witnesses the consequent. Therefore $G_g \models \varphi_i$, and thus $G_s \models_k \varphi_i$. □

Analysis of ApxIChase. We prove that ApxIChase terminates and returns a feasible top-K grounded witness pair set as a $(1-1/e)$-approximation to the current optimum over the grounded witness pairs available at the request time.

Correctness. The algorithm executes at most L rounds(line 3). In each round $l \in [1, L]$, L-Chase (line 6) performs bounded pattern matching over $G^L(v_t)$ without edge insertion, terminating when all source constraints Σ_s are processed. L-Backchase (line 12) enforces at most k new edges per applicable inverse constraint $\varphi^{-1} \in \Sigma_b$, as tracked by budget $\mathcal{M}[i].b$(Figure 9, line 3), since $G^L(v_t)$ is finite and each edge insertion reduces the available budget, the backchase phase terminates after at most $|\Sigma| \cdot k$ edge insertions. The outer loop completes in bounded steps and guarantees termination.

Each grounded witness pair admitted to W_K satisfies two conditions: (1) VerifyWitness ensures that every G_s is either factual or counterfactual, (2) In L-Backchase, for each $\varphi^{-1} = (c, P)$, the algorithm inserts a minimal edge set ΔE to enforce at least one match of P, making the witness G_s satisfy $G_s \models_k \varphi$. The memoization map $\mathcal{M}$ records lower bounds on $|\Delta E|$, ensuring minimality. Consequently, each grounded graph $G_g \in \mathcal{G}_g$, together with its corresponding witness G_s, forms a feasible k-grounded witness pair.

Approximability. Let C denote the candidate pair pool of grounded witness pairs generated so far. Upon request time, UpdateWK greedily refreshes the current top-K set by selecting witness pairs from C according to their marginal gain under the set objective $F(\mathcal{P})$. This yields a $(1-1/e)$-approximation guarantee under our set objective, formalized in Theorem 3.

Proof. At any request time, let C denote the set of candidate grounded witness pairs generated so far, where each element is a pair (G_s, G_g). Consider the cardinality-constrained maximization problem $\max_{\substack{\mathcal{P}\subseteq C \\ |\mathcal{P}|\le K}} F(\mathcal{P})$, where, by definition, $F(\mathcal{P}) = \sum_{(G_s,G_g)\in\mathcal{P}} q(G_s, G_g) + \gamma \cdot \mathrm{cov}(\mathcal{P})$. We show that $F(\cdot)$ is a monotone submodular set function. First, the term $\sum_{(G_s,G_g)\in\mathcal{P}} q(G_s, G_g)$ is modular, since it is a sum of fixed per-witness scores that depend only on individual pairs. Second, the term $\mathrm{cov}(\mathcal{P}) = \frac{\left|\bigcup_{(G_s,G_g)\in\mathcal{P}} \Sigma'(G_g)\right|}{|\Sigma|}$ is a normalized set-coverage function, where $\Sigma'(G_g) \subseteq \Sigma$ denotes the subset of constraints grounded by G_g. Since set coverage is monotone submodular, $\mathrm{cov}(\mathcal{P})$ is monotone submodular as well. Therefore, $F(\mathcal{P})$, being the sum of a modular function and a monotone submodular function, is itself monotone submodular. Under the cardinality constraint $|\mathcal{P}| \le K$, the standard result of Nemhauser et al. [34] implies that the greedy algorithm achieves a $(1-1/e)$-approximation to the optimum. Since UpdateWK constructs W_K by repeatedly selecting the grounded witness pair with the largest marginal gain under $F(\cdot)$ from the candidate pool generated so far, at any request time it returns a set W_K such that $F(W_K) \ge (1-1/e) \cdot F(\mathcal{P}^*)$, where $\mathcal{P}^* \in \arg\max_{\substack{\mathcal{P}\subseteq C \\ |\mathcal{P}|\le K}} F(\mathcal{P})$. Hence, UpdateWK achieves a $(1-1/e)$-approximation ratio with respect to the current optimum obtainable from the grounded witness pairs generated so far. □

Cost Analysis. The delay time per request is $O(|\Sigma|^2 \cdot |G^L(v_t)| + L|\Sigma|^2 + |\Sigma||C|K)$. The first term $|\Sigma|^2|G^L(v_t)|$ arises from a double loop: L-Chase generates $O(|\Sigma||G^L(v_t)|)$ candidate witnesses, each of which is checked against all $|\Sigma|$ constraints to build Σ_b via the memoization map $\mathcal{M}$. The $L|\Sigma|^2$ term captures the bounded L-round constraint propagation and the interactions among inverse constraints during L-Backchase, where enforcing φ^{-1} may trigger updates to the corresponding budget entries $\mathcal{M}[i].b$. The $|\Sigma||C|K$ term reflects the request-time greedy refresh performed by UpdateWK: at most K selection rounds are executed, and in each round every remaining candidate pair in the current pool C is scanned once to compute its marginal gain. Under a direct implementation, evaluating the coverage increment in $\Delta((G_s, G_g) \mid W_K) = q(G_s, G_g) + \gamma \cdot \frac{|\Sigma'(G_g) \setminus \mathrm{CovSet}(W_K)|}{|\Sigma|}$ takes $O(|\Sigma|)$ time, yielding the stated bound.

The memory cost per request is $O\Big(|G^L(v_t)| + |G_\Sigma| + |\Sigma| \cdot (p+2) + |C| + K \cdot (|G^L(v_t)| + |\Sigma|k)\Big)$. The $|G^L(v_t)|$ term accounts for storing the L-hop subgraph (and a constant number of working copies) accessed by L-Chase and L-Backchase. The $|G_\Sigma|$ term is for the union pattern graph of all constraints in Σ. The $|\Sigma| \cdot (p+2)$ term is for the memoization map $\mathcal{M}$, whose size is $|\Sigma| \times (p+2)$, where each entry keeps a length-$(p+1)$ pattern-status vector $\mathcal{M}[i].P$ and an additional budget/lower-bound value $\mathcal{M}[i].b$. The $|C|$ term accounts for the metadata of the current candidate pair pool maintained for request-time top-K selection. Finally, the $K \cdot (|G^L(v_t)| + |\Sigma|k)$ term reflects storing the selected top-K grounded witness pairs in W_K, where each pair consists of a witness bounded by the receptive field and a corresponding grounded graph with at most $|\Sigma|k$ enforced edges introduced during L-Backchase.

Analysis of HeuIChase. We present the detailed analysis below.

Correctness. We analyze HeuIChase under the restricted tree/arborescence semantics adopted in Section 6: each returned witness G_s is a verified tree witness rooted at v_t, and each returned grounded graph G_g is a weighted arborescence of $G^L(v_t)$ rooted at v_t and containing G_s. Grounding in HeuIChase is realized by supporting-arborescence expansion, rather than by explicitly constructing a minimum insertion set ΔE.

Lemma 8: *Every pair (G_s, G_g) returned by* HeuIChase *is a feasible grounded witness pair in the tree/arborescence setting.* □

Proof. Let (G_s, G_g) be a pair returned by HeuIChase .

(1) **Witness Correctness.** Procedure L-Chase generates candidate witnesses by matching a tree pattern P against sampled trees from $G^L(v_t)$, and retains a candidate only if it passes VerifyWitness. Hence every returned G_s is a factual or counterfactual witness for $Q(M, v_t, G)$. By construction of HeuIChase, G_s is a rooted tree containing v_t.

(2) **Containment and Arborescence Structure.** Given a verified witness G_s, HeuIChase contracts G_s into a supernode s and applies Edmonds' algorithm on the contracted graph. Edmonds' algorithm returns a maximum-weight arborescence rooted at s over

the contracted graph. Expanding the supernode s back to the original witness edges yields a connected rooted arborescence G_g in the original graph domain. Since the contraction preserves the internal structure of G_s, we have $G_s \subseteq G_g$. Moreover, since G_s is rooted at v_t, the expanded arborescence G_g is rooted at v_t in the original graph.

(3) **Grounding Correctness.** Let $\Sigma'(G_g) = \{\varphi \in \Sigma \mid G_g \models \varphi\}$ be the set of constraints grounded by G_g. After constructing G_g, HeuIChase evaluates which constraints in Σ are satisfied by the resulting arborescence and retains the pair only when $\Sigma'(G_g) \neq \emptyset$. For every $\varphi = (P, c) \in \Sigma'(G_g)$, the graph G_g contains a co-occurring match of P and c under the same variable assignment; hence G_g grounds G_s under $\Sigma'(G_g)$.

Therefore, G_s is a verified witness, G_g is a rooted arborescence containing G_s, and G_g grounds G_s under a nonempty subset of constraints. Thus (G_s, G_g) is a feasible grounded witness pair in the tree/arborescence setting. □

Remark. The above soundness result is intentionally weaker than the explicit k-grounded semantics used by ApxIChase. Unlike ApxIChase, HeuIChase does not explicitly materialize a minimum insertion set ΔE or guarantee budget-sensitive repair. Instead, it heuristically realizes L-Backchase by expanding a verified tree witness into a supporting weighted arborescence.

Approximability. HeuIChase adopts the same UpdateWK routine as ApxIChase. Let C denote the candidate pair pool of grounded witness pairs generated by HeuIChase and available at request time.

Proposition 9. *Let $\mathcal{P}^* \in \arg\max_{\mathcal{P}' \subseteq C, |\mathcal{P}'| \leq K} F(\mathcal{P}')$ be an optimal size-K witness-pair set over the current candidate pool C. Then* UpdateWK *returns a set W_K such that $F(W_K) \geq (1 - 1/e) \cdot F(\mathcal{P}^*)$.*

Proof. For each candidate pair $(G_s, G_g) \in C$, the score $q(G_s, G_g)$ is fixed once the pair is generated; for HeuIChase, this score is instantiated by witness conciseness. Hence the term $\sum_{(G_s,G_g)\in\mathcal{P}} q(G_s, G_g)$ is modular over the candidate set. Moreover, $\mathrm{cov}(\mathcal{P}) = \frac{\left|\bigcup_{(G_s,G_g)\in\mathcal{P}} \Sigma'(G_g)\right|}{|\Sigma|}$ is a normalized set-coverage function, and is therefore monotone submodular. Thus the overall set objective $F(\mathcal{P}) = \sum_{(G_s,G_g)\in\mathcal{P}} q(G_s, G_g) + \gamma \cdot \mathrm{cov}(\mathcal{P})$ is monotone submodular over C. Under the cardinality constraint $|\mathcal{P}| \leq K$, the classical greedy result of Nemhauser et al. implies that greedy maximization achieves a $(1 - 1/e)$-approximation to the optimum over C. Since UpdateWK greedily refreshes the top-K set by repeatedly selecting the pair with the largest marginal gain under $F(\mathcal{P})$, the bound follows. □

Remark. This approximation guarantee is conditional on the candidate pool C generated by HeuIChase. Since tree sampling and weighted arborescence construction are heuristic, the guarantee does not imply a constant-factor approximation to the global optimum over all feasible grounded witness pairs.

Cost Analysis. The complexity reduction stems from two substitutions that avoid exhaustive subgraph enumeration.

(1) **Tree Pattern Matching.** Matching a tree pattern P of size p against m sampled trees costs $O\big(m|G^L(v_t)| + mp^2\big)$ per round when the sampled trees are reused across constraints. The first term accounts for sampling m spanning trees from $G^L(v_t)$, while the second term captures tree-pattern matching checks. Across all constraints, the per-round matching cost is $O\big(m|G^L(v_t)| + m|\Sigma|p^2\big)$. Over L rounds, this yields $O\big(Lm|G^L(v_t)| + Lm|\Sigma|p^2\big)$. When p is treated as a small constant, this simplifies to $O\big(Lm|G^L(v_t)| + Lm|\Sigma|\big)$.

(2) **Weighted Arborescence Computation.** For each retained witness candidate G_s, HeuIChase contracts G_s into a supernode and invokes Edmonds' algorithm once on the contracted graph. Under the implementation adopted in our experiments, each Edmonds run costs $O\big(|G^L(v_t)| \log |G^L(v_t)|\big)$. If $|C|$ candidate pairs have been generated up to request time, the total arborescence-construction cost is $O\big(|C|\,|G^L(v_t)| \log |G^L(v_t)|\big)$. In addition, grounding validation on each constructed arborescence requires checking which constraints in Σ are satisfied by the resulting tree. Since each constraint pattern is a tree of bounded size p, this takes $O(|\Sigma|p)$ per candidate, which is absorbed into candidate maintenance when p is treated as a small constant.

At request time, the top-K selection incurs the same greedy refresh cost as in ApxIChase, namely $O\big(|\Sigma|\,|C|\,K\big)$, over the current candidate pair pool C. Therefore, the overall request-time cost of HeuIChase is $O\Big(Lm|G^L(v_t)| + Lm|\Sigma|p^2 + |C|\,|G^L(v_t)| \log |G^L(v_t)| + |\Sigma|\,|C|\,K\Big)$. When p is bounded by a small constant, this simplifies to $O\Big(Lm|G^L(v_t)| + Lm|\Sigma| + |C|\,|G^L(v_t)| \log |G^L(v_t)| + |\Sigma|\,|C|\,K\Big)$.

In practice, m is a small constant (*e.g.*, $m = 5$ in our experiments), so the main saving over ApxIChase comes from replacing general subgraph enumeration by tree matching and replacing explicit backchase repair by a single arborescence expansion step per retained witness candidate.

The memory cost depends on how candidate pairs are materialized.

Materialized Implementation. If sampled trees and candidate pairs are stored explicitly, the memory cost per request is $O\big(|G^L(v_t)| + m|G^L(v_t)| + |C|\,|G^L(v_t)| + K|G^L(v_t)|\big)$. The four terms correspond to storing the L-hop neighborhood, the sampled trees, the current candidate pair pool, and the selected top-K witness pairs, respectively.

Compressed Implementation. If candidate pairs are stored only through parent pointers, scores, and grounded-constraint identifiers, the bookkeeping term reduces to $O(|C|)$ on top of shared graph storage, yielding $O\big(|G^L(v_t)| + m|G^L(v_t)| + |C| + K|G^L(v_t)|\big)$. If sampled trees are generated and processed on-the-fly, the term $m|G^L(v_t)|$ further reduces to $O(|G^L(v_t)|)$ working memory. Unlike ApxIChase, no additional $O(|\Sigma|k)$ space is needed, since HeuIChase does not explicitly maintain grounded-edge insertion sets or budget-tracking structures for individual candidates.

9.3 Appendix C: Complementary Experimental Study and Additional Discussion

Discussion: Applications of Grounded Witnesses. We briefly discuss why k-grounded witnesses are useful in practice. (1) High interpretability. It readily suggests critical fraction G_s of the original graph G that are responsible to $Q(M, v_t, G)$, and an extended counterpart G_g to ensure a co-occurrence of matches of P and c, hence

Dataset	#rules (before → after)	Avg $\|P\|$ size	Active ratio
DBLP	64 → 20	3.4	0.54
Cora	25 → 20	3	0.46
MUTAG	9 → 9	3.56	0.69
BAShape	5 → 5	2.8	0.60
ATLAS	28 → 12	3.75	0.61
ogbn-Papers100M	52 → 20	3.35	0.23
Tree-Cycles	18 → 14	3.14	0.58

Table 3: Constraint statistics across datasets.

Algorithm 5 : paraIChase: Parallel Witness Generation

Input: Graph G, constraint set Σ, model M, target node set V_T, integers L,K,k, #workers w, ALG $\in$ {ApxC, HeuC, Exh, GEX, PGX}
Output: $\{(v_t, W_K(v_t)) \mid v_t \in V_T\}$

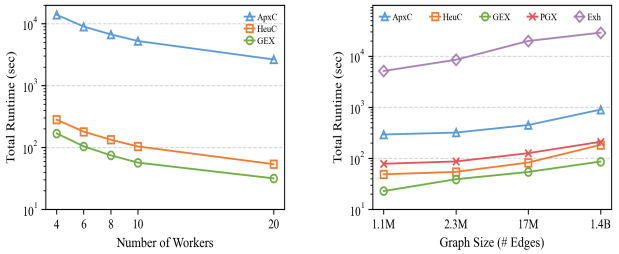
Coordinator computes $\mathcal{G}^L(V_T) = \{(v_t, G^L(v_t)) \mid v_t \in V_T\}$
$\Pi \leftarrow$ LoadBalance$(\mathcal{G}^L(V_T), w)$ ▷ load balance by $|G^L(v_t)|$
for $i = 1$ **to** w **do in parallel**
  $W_K^{(i)} \leftarrow \emptyset$
  for all $(v_t, G^L(v_t)) \in \Pi_i$ **do**
    Execute ALG on $(v_t, G^L(v_t))$ to obtain $W_K(v_t)$
    $W_K^{(i)} \leftarrow W_K^{(i)} \cup \{(v_t, W_K(v_t))\}$
return $\bigcup_{i=1}^{w} W_K^{(i)}$

Figure 11: Algorithm paraIChase**: Parallelized witness generation framework with load-balanced subgraph distribution.**

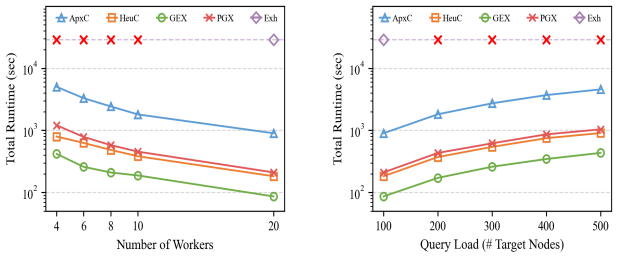
directly grounding G_s to genuine, real-world data evidence as specified by φ. The constraint $\varphi \in \Sigma$, in turn, contributes to rule-based GNN interpretation as logical rules. (2) Data preparing and refinement for graph learning. While conventional graph enrichment focuses on maximizing data completeness rather than "ML-aware", the additional edges ΔE readily suggest "how" G should be modified to include missing data, that are specifically relevant to the decision making process of M. This also indicates useful training and testing triples for GNNs. (3) Adversarial graph learning. The ΔE and G_s together indicate "vulnerable" area that may be attacked or poisoned to affect GNN M's output, hence robust training can be localized to avoid expensive defense cost.

Constraint Statistics. Table 3 summarizes the basic statistics of the constraint pools used in our experiments across datasets. Specifically, "before → after" reports the number of constraints before and after pruning, "Avg $|P|$ size" denotes the average number of antecedent edges in the final rule pool, and "Active ratio" is the average fraction of active constraints per workload over the final rule pool. These statistics show that the final constraint pools remain compact after pruning while still retaining a nontrivial fraction of active constraints for grounding. They also indicate that the antecedent patterns used in practice are local and moderate in size, which is consistent with our focus on bounded witness grounding over local graph neighborhoods.

Parallel Algorithm. Fig. 11 presents paraIChase, a parallel witness generation framework. A coordinator process induces L-hop subgraphs $\{(v_t, G^L(v_t)) | v_t \in V_T\}$ and partitions them across w workers via First-Fit Decreasing bin-packing based on subgraph



(a) Runtime Varying Number of Workers on OGBN-Papers100M **(b) Runtime Varying Graph Size on Tree-Cycles**



(c) Runtime Varying Number of Workers on Tree-Cycles **(d) Runtime Varying Query Load on Tree-Cycles**

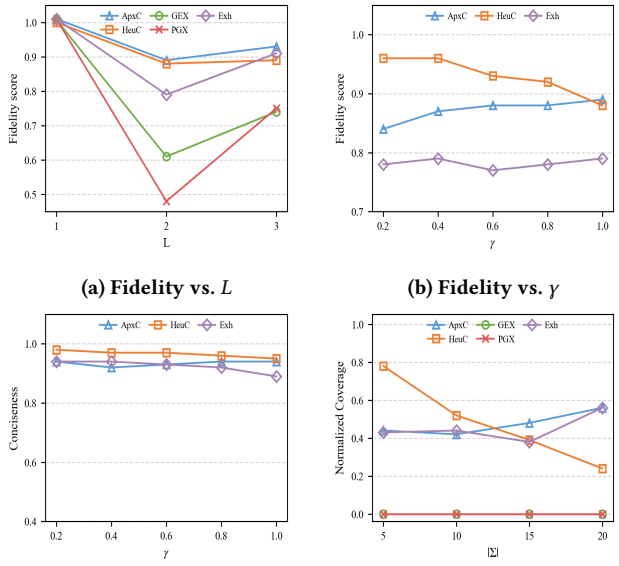
Figure 12: Scalability Test

size $|G^L(v_t)|$, using a min-heap over worker loads to mitigate stragglers. Each worker independently runs one of the base algorithms ALG $\in$ {ApxC, HeuC, Exh, GEX, PGX} on its assigned subgraphs. Global parameters are shared; workers access subgraphs via indexed slicing to avoid duplication. Verification is performed in mini-batches of size 32 per worker to amortize inference costs. The coordinator aggregates per-target results $W_K(v_t)$ to compute final metrics.

Exp-3: Scalability. We report the scalability of our parallel framework (paraIChase) using both real and synthetic graphs. In all the tests, L=2, k=2, K=6, and we use 100 target nodes by default.

OGBN-Papers100M. Figure 12(a) shows that both ApxIChase and HeuIChase achieve clear speedups when scaling from 4 to 20 workers. Despite highly unbalanced subgraph sizes (some >80K edges), runtimes remain tolerable—dropping from 13.9K→2.6K s for ApxIChase and 282 → 54 s for HeuIChase. PGExplainer and naiveChase fail to scale, while GNNExplainer remains fastest but saturates beyond 10 workers.

Tree-Cycles. We further evaluate scalability on the synthetic Tree-Cycles dataset under three settings. (1)*Varying graph size:* As the graph grows from 1.1M to 1.4B edges (≈25 GB), both ApxIChase and HeuIChase show moderate runtime growth and remain tractable, while naiveChase exceeds 28,800 s (>8 h) with 20 workers and causes severe memory inflation. Each processor handles 4–6 L-hop subgraphs (≈4.6K edges). Under naiveChase, enforcing all constraints can double or triple subgraph size, peaking at ≈1 GB per processor (≈20 GB total). In contrast, ApxIChase and HeuIChase only operate on compact witnesses—typically 20–40% of subgraph size—keeping per-processor memory below 200–300 MB. Overall, they reduce memory by 4–5× versus naiveChase while maintaining



(a) Fidelity vs. L **(b) Fidelity vs. γ**

(c) Conciseness vs. γ **(d) Coverage vs. $|\Sigma|$**

Figure 13: Complementary Experiments

stable runtime and balanced load. (2)*Varying processor count:* Fixing the 1.4B-edge graph and 100 target nodes, we vary workers from 4 to 20 (Fig. 12(c)). Both ApxIChase and HeuIChase achieve sublinear yet consistent speedups, limited mainly by coordination and I/O during extraction and aggregation. The makespan decreases by over 5×, showing effective load balance and low communication contention under high parallelism. (3)*Varying query load:* On the same graph with 20 workers, increasing target nodes from 100 to 500 (Fig. 12(d)) yields nearly linear runtime growth, dominated by extraction and grounding, with consistent trends across algorithms. Overall, our methods achieve 1–2 orders of magnitude runtime improvement and 4–5× lower memory than the exhaustive naiveChase, maintaining stable scalability across graph size, worker count, and query load.

Exp-4: Complementary Experiments on DBLP. We conduct several complementary experiments on the *DBLP* dataset to further examine the sensitivity of our methods to model depth, objective weighting, and constraint pool size.

Fidelity vs. L. We vary the model depth $L \in \{1, 2, 3\}$ while keeping all other factors fixed. As shown in Fig. 13(a), our methods maintain relatively high fidelity as L increases, although all methods exhibit some drop from L=1 to larger neighborhoods. In particular, ApxIChase and HeuIChase remain consistently more faithful than GNNExplainer and PGExplainer at L=2 and L=3. This suggests that incorporating grounding into witness generation does not prevent the selected witnesses from preserving the model's prediction behavior, even when the receptive field becomes larger.

Fidelity vs. γ. We vary $\gamma \in \{0.2, 0.4, 0.6, 0.8, 1\}$ while maintaining $\alpha = \beta = \frac{1}{2}$. As shown in Fig. 13(b), the fidelity of ApxIChase increases slightly as γ grows, whereas HeuIChase shows a mild decline. Meanwhile, naiveChase remains largely stable across different values of γ. Overall, these results indicate that increasing the weight of coverage in the set objective does not cause a drastic loss of fidelity, although different witness-generation strategies respond differently to this trade-off.

Conciseness vs. γ. Fig. 13(c) shows that conciseness remains high and largely stable as γ varies. Both ApxIChase and HeuIChase maintain consistently high scores across all settings. In particular, HeuIChase remains slightly higher, indicating that although its arborescence expansion may produce a larger grounded graph, the selected witness itself is still compact. By contrast, naiveChase shows a mild decline as γ increases, suggesting that placing more emphasis on coverage may introduce slightly larger witnesses. We report only our methods here, since γ is specific to our objective.

Coverage vs. $|\Sigma|$. We further vary the size of the constraint pool, with $|\Sigma| \in \{5, 10, 15, 20\}$, and report normalized coverage in Fig. 13(d). Overall, the baselines remain at zero, while our methods achieve substantially higher coverage. For ApxIChase and naiveChase, the trend is not strictly monotone: enlarging the constraint pool changes both the number of active constraints and the subset that can be covered, so the normalized coverage may increase or decrease depending on the specific pool. In contrast, HeuIChase decreases steadily as $|\Sigma|$ grows, indicating that its tree-biased candidates cover a shrinking fraction of the enlarged constraint pool. This result also highlights why we report normalized coverage rather than raw counts: the full constraint pool may contain many constraints that are irrelevant to the current workload.

	GNNExplainer			PGExplainer		
Dataset	**Before**	**After**	**Δ**	**Before**	**After**	**Δ**
MUTAG	0.200	0.657	**+0.457**	0.180	0.571	**+0.391**
DBLP	0.000	0.240	**+0.240**	0.000	0.320	**+0.320**
ATLAS	0.170	0.540	**+0.370**	0.310	0.480	**+0.170**
Cora	0.000	0.596	**+0.596**	0.000	0.445	**+0.445**

Table 4: Effect of grounding on baseline explainers. Δ denotes the absolute increase in normalized coverage after grounding.

Exp-5: Effect of Grounding on Baseline Explainers. To further examine the contribution of the grounding stage, we compare the normalized coverage of two representative baseline explainers, GNNExplainer and PGExplainer, before and after applying the same grounding process used in our framework. For each explainer, the "Before" column reports the coverage of its raw witness, while the "After" column reports the coverage obtained after feeding the same witness into our grounding stage under the default experimental configuration. As shown in Table 4, grounding consistently improves the coverage of both baseline explainers across four datasets. Grounding thus accounts for a substantial part of the final coverage gain, while the quality of the input witness still influences the resulting coverage after grounding.